CH04 CSS传统布局和Flex布局

CSS布局研究的是页面中的盒子怎样排列、怎样确定位置，以及怎样共同使用可用空间。CH03已经介绍了盒模型、尺寸和基本显示行为，本章进一步讨论这些盒子之间的排列关系。

本章分为五部分。首先介绍正常文档流及其默认排列规则；然后介绍浮动的图文环绕需求、浮动规则与清除浮动；接着介绍定位方式、定位参照与覆盖关系；第四部分介绍Flexbox的容器、项目、轴、对齐和空间分配；最后将这些知识组合成校园活动页面。各部分在概念和规则之后提供相应案例，完整页面将在CH05继续扩展为Grid与响应式布局。

1 正常文档流

1.1 什么是正常文档流

正常文档流是浏览器在没有浮动、绝对定位等改变常规排列的条件下，组织文档中块级盒子和行内内容的基本布局过程。页面内容首先具有HTML所表达的结构和顺序，浏览器再根据盒子的显示类型、尺寸、边距和可用空间安排位置。

正常流主要包含两类排列关系：块级盒子在块布局中依次排列；文字、链接等行内内容在行框中排列。对本章采用的水平书写页面来说，前者通常表现为上下排列，后者通常表现为一行从左向右、空间不足后换行。

图4-1 什么是正常文档流

“流”表达了内容之间的连续关系。前面的段落增加文字，高度随之增大，后面的段落会自然向下移动；一行容纳不下更多文字，后续文字便进入下一行。浏览器会随着内容与可用宽度重新计算，不要求为每段正文分别指定坐标。

图4-2 前面的内容变高，后面的内容自然下移

正常流并不表示所有元素都上下排列，也不表示每个元素都占据相同大小的矩形。理解默认布局，需要进一步区分块级与行内行为。

1.2 块级盒子与行内内容

在普通块布局中，块级盒子通常沿块方向依次排列。水平书写时，一个普通块盒子从前一个块盒子下方开始；宽度为`auto`时，会结合内外边距与边框使用可用宽度。

`section`、`div`、标题和段落等通常具有块级的默认显示行为。即使把某个块盒子的宽度设小，后面的普通块盒子通常也不会自动补到它右边，因为缩小尺寸没有改变块布局规则。

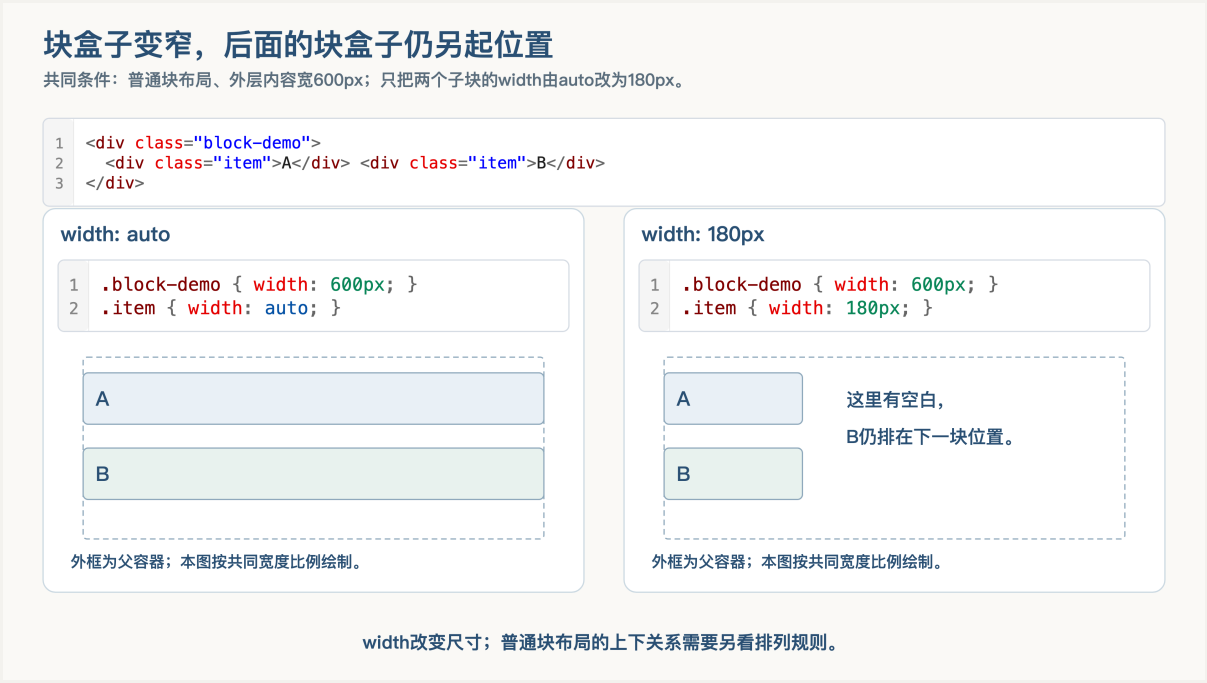

图4-3 块盒子变窄，后面的块盒子仍另起位置

行内内容则进入所在容器的行框。普通文字、`span`和链接等可以在一行中连续排列，浏览器根据字体、内容和可用宽度决定断行位置。

```
<p>欢迎参加<strong>校园读书交流会</strong>。</p>
<p>请先阅读<a href="#registration">参与方式</a>。</p>
```

这两段正文形成两个块级区域；`strong`与`a`参与各自段落内部的行内排版。HTML源码中的换行也不直接等于页面中的换行，浏览器会按布局规则组织内容。

图4-4 段落之间上下排，段落内部沿行排

因此，同一个页面可以同时存在“段落之间上下排列”和“段落内部文字沿行排列”。前者与后者共同构成正常流的基本认识。

1.3 display与基本显示行为

元素的语义由HTML表达，盒子的显示行为可以由CSS的`display`调整。常见值包括：

值	基本行为	典型用途
`block`	按块级方式参与外部排列	图片独立成行、组织内容区域
`inline`	普通文本盒子参与行内排版	段落中的短文字
`inline-block`	整体参与行内排列，内部可按块容器组织内容	带尺寸和留白的导航链接
`none`	不生成布局盒子，也不保留相应空间	按实际需要隐藏内容

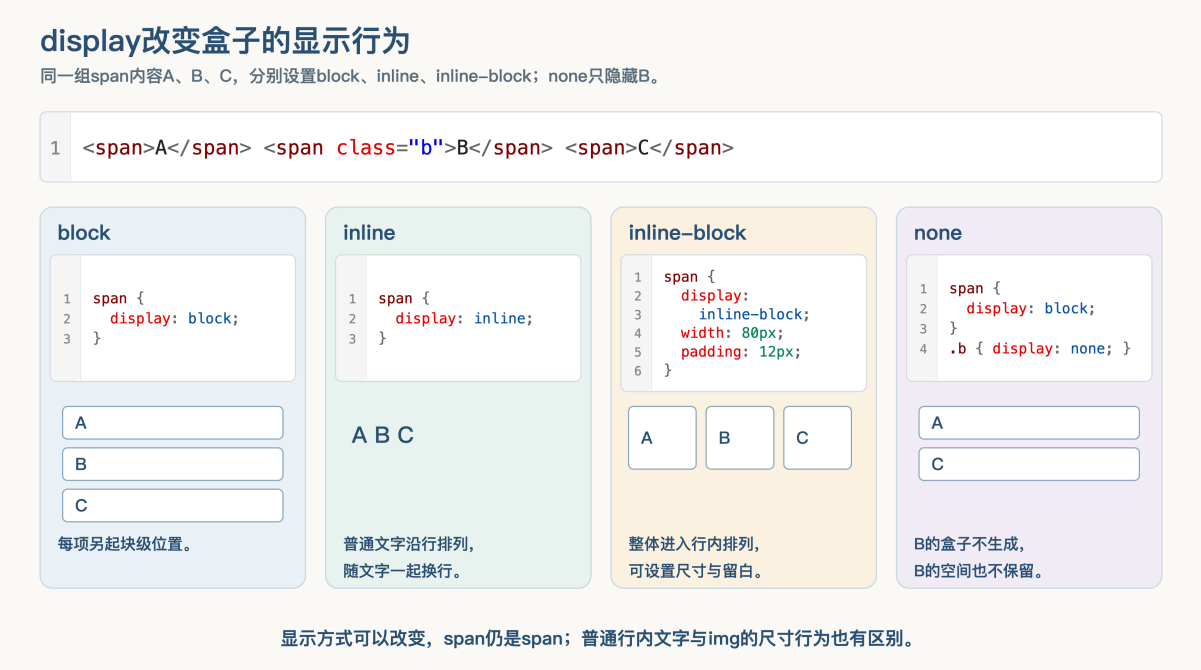

图4-5 display改变盒子的显示行为

图4-6 display:none让盒子与相应空间一起消失

`inline-block`同时具有两方面特征：从外部看，它作为一个整体放进行内排列；从内部看，它可以容纳内容并使用宽高、边框和内边距。前一个项目旁边还有空间时，后一个项目可以继续排在这一行。

普通非替换行内盒子与`inline-block`的尺寸行为不同；图片又属于替换元素，不能把“行内元素不能设置宽高”作为通用结论。CH03已经介绍了这些区别。

图4-7 inline-block：外部沿行排，内部可以设置尺寸

改变`display`不会改写HTML语义。链接采用`block`以后仍然是链接，标题采用`inline-block`以后也仍然是标题。布局需要同时考虑内容角色与显示方式。

1.4 案例 用正常流组织活动通知

下面用标题、正文与导航组成一个活动通知区域：

```
<section class="flow-notice">
  <h2>校园读书交流会</h2>
  <p>带一本喜欢的书，分享一次阅读体验。</p>
  <p>活动安排与参与方式见下方链接。</p>
  <nav class="flow-nav" aria-label="通知导航">
    <a href="#schedule">活动安排</a>
    <a href="#registration">参与方式</a>
  </nav>
</section>
```

图4-8 活动通知：先看HTML怎样组织内容

```
.flow-notice {
  max-width: 640px;
  padding: 24px;
  border: 1px solid #cbd5e1;
}
```

```css
.flow-notice h2,
.flow-notice p {
  margin: 0 0 16px;
}

.flow-nav a {
  display: inline-block;
  margin: 4px;
  padding: 6px 12px;
  border: 1px solid #205493;
}
```

标题、两个段落与导航区域按正常块布局依次排列，外层没有指定高度，因此正文增加后，整个通知会自然向下增长。段落内部的文字根据可用宽度形成一行或多行。

导航链接采用`inline-block`，可以带着内边距和边框作为整体排在行内。空间不足时，后一个链接进入下一行；相邻标签之间的源码空白也可能形成可见间隙。

图4-9 活动通知：完整CSS与正常流结果

如果只把某个段落改为`width: 180px`，它会变窄，但后面的段落仍然另起块级位置。如果给通知固定一个过小高度，内容则可能超出边框。这两个变化分别说明：宽度不会单独改变排列方式，高度也应当与内容增长的需要相适应。

2 浮动与图文环绕

2.1 为什么需要浮动

正常流已经能组织连续正文，但图文排版还有一种常见需求：图片放在段落一侧，前几行文字沿图片旁边排列，超过图片高度后，后面的文字重新使用整行宽度。这种效果称为图文环绕。

只把图片宽度设小，并不能让后面的块级段落自动变成旁边的一列；给图片绝对指定坐标，也没有表达文字应避开图片的关系。CSS的浮动机制为这种排版提供了专门规则。

浮动让盒子向所在区域的一侧移动，同时使附近正常流中的文字行避开它的外边距范围。它脱离普通正常流盒子的常规排列，却仍会影响周围行内内容的可用空间。

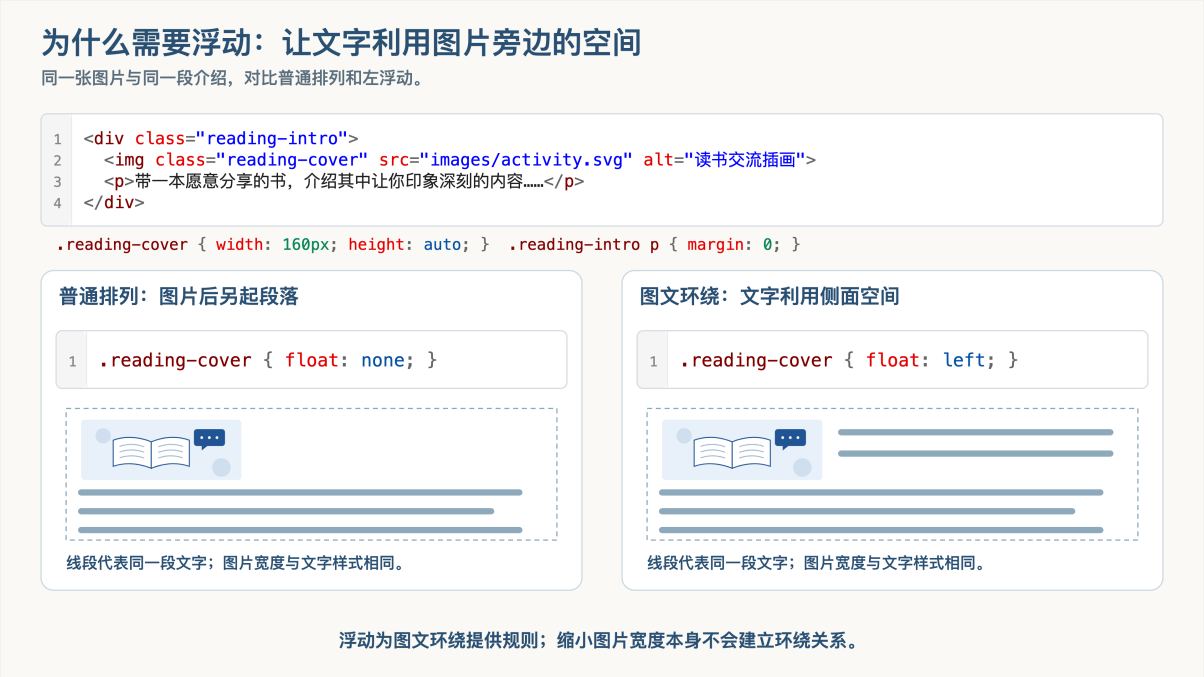

图4-10 为什么需要浮动：让文字利用图片旁边的空间

浮动因此既不同于普通块布局，也不同于后面要学习的绝对定位。图文环绕是理解它的合适起点。过去浮动也被广泛用于页面分栏；现在组织一组项目和明确行列时，通常可以使用Flexbox或Grid，而浮动仍适合文字环绕等需求。

2.2 浮动的基本概念和用法

`float`用于选择浮动方向，常用值如下：

值	含义
`none`	不浮动，采用该盒子原本的布局规则
`left`	向左侧浮动
`right`	向右侧浮动

浮动元素会尽量向指定一侧排列，并考虑容器边界及前面浮动的位置。这里的“侧”应结合所在布局区域理解，不是把元素移动到浏览器窗口边缘。

图4-11 float的三个常用值：none、left、right

浮动盒子的尺寸、内外边距和边框都会影响周围可用空间。用于图文环绕的图片通常设置明确宽度，并在靠近文字的一侧留出外边距：

```
.reading-cover {
  float: left;
  width: 160px;
  height: auto;
  margin: 0 16px 8px 0;
}
```

这组规则使图片浮动到左侧，按原比例显示，并在右侧和下方分别留出16px、8px的距离。文字行在图片旁边使用剩余宽度；超过浮动外边距范围后，再恢复较完整的行宽。

改为右浮动时，应同时检查留白方向；改为`none`时，图片恢复为非浮动内容。取消浮动与让后续段落避开浮动，是两个不同操作。

2.3 浮动带来的影响与基本处理

浮动除了改变文字的排列，还需要关注后续内容从哪里开始，以及外层容器能否包住图片。这是两个相关但不同的问题。

图文介绍后面可能有一句总结“欢迎带着问题参加”。如果它也放在浮动图片之后，就可能继续沿图片旁边排列。希望这句话从图片下方另起时，可以给它设置`clear`：

```
<div class="reading-intro">
  <img class="reading-cover" src="images/activity.svg"
       alt="读书交流插画" width="800" height="300">
  <p>选择一本喜欢的书。</p>
  <p class="clear-summary">欢迎带着问题参加。</p>
</div>
```

图片沿用第2.2节的左浮动规则；为便于比较总结的位置，下面的对照还将段落的默认外边距设为零：

```
.reading-intro p {
  margin: 0;
}

.clear-summary {
  clear: both;
}
```

`clear: both`写在总结段落上，使它在需要时向下避开前面的浮动及其外边距，从图片下方开始。图片仍然左浮动，前面的介绍仍然可以环绕；只有把图片自己的`float`改成`none`，才是取消图片的浮动。

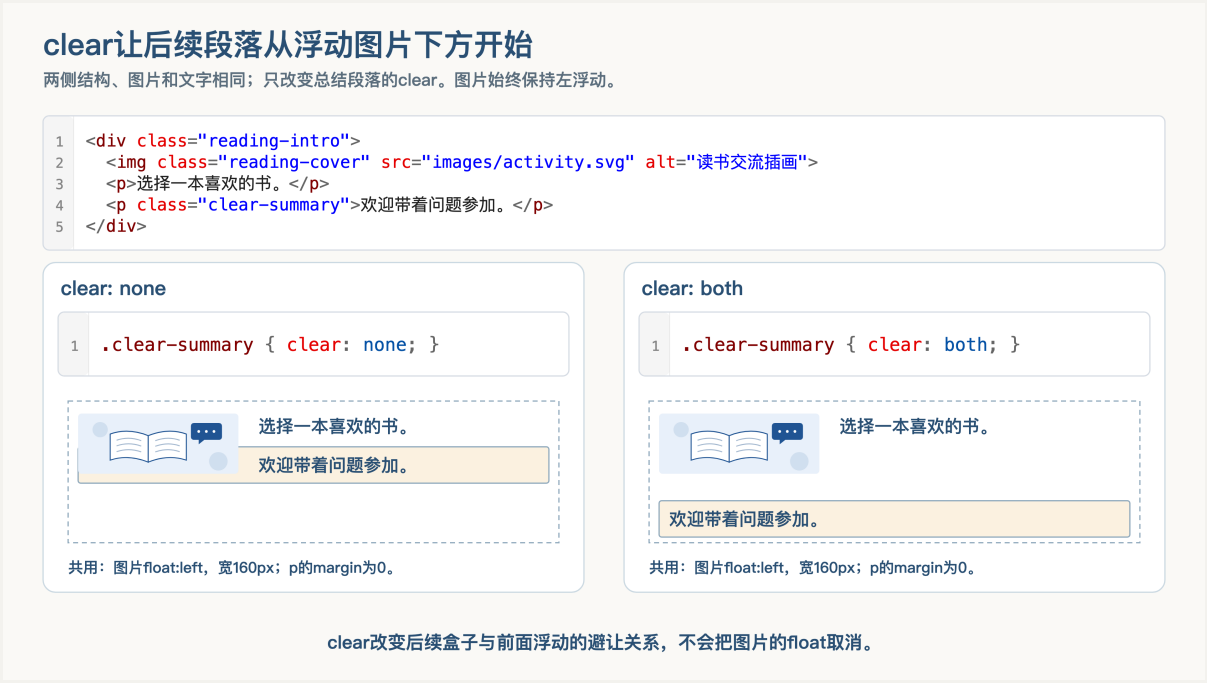

图4-12 clear让后续段落从浮动图片下方开始

另一个问题发生在父容器上。在没有总结段落的图文区域中，如果介绍文字很少而浮动图片较高，未设置高度的普通块容器就可能包不住图片。这是因为浮动图片不像普通正常流内容那样撑开这个容器，图片下方的一部分可能伸到边框外。

希望整个图文区域包含图片，并让外面的内容排在整个区域之后，可以在图片和文字的共同父元素上设置：

```
.reading-intro {
  display: flow-root;
}
```

`flow-root`使父容器的自动高度能够包含内部浮动，同时保留里面的图文环绕。因此，`clear`设置在需要避让的后续内容上，`flow-root`设置在需要包含浮动的父容器上，应按具体需求选择。段落盒子边界、容器高度计算及进一步对照见附录D。

2.4 案例 实现活动介绍的图文环绕

活动介绍需要把插画放在左侧，让正文环绕，并使最后一句提示从图片下方开始。完整结构如下：

```html
<div class="reading-intro">
  <img class="reading-cover" src="images/activity.svg"
       alt="展开的书与交流气泡的插画" width="800" height="300">
  <p>带一本愿意分享的书，介绍其中让你印象深刻的内容。</p>
  <p>可以讲一个人物、一段经历，也可以提出一个仍想继续讨论的问题。</p>
  <p class="clear-summary">欢迎带着问题参加。</p>
</div>
```

图4-13 活动介绍案例：先看图片、正文与总结的结构

```css
.reading-intro {
  display: flow-root;
}

.reading-cover {
  float: left;
  width: 160px;
  max-width: 45%;
  height: auto;
  margin: 0 16px 8px 0;
}

.clear-summary {
  clear: both;
}
```

图片的`float: left`负责环绕关系，右侧外边距让文字与图像分开。图片宽度还受父容器45%的最大宽度限制，较窄时可以继续缩小，自动高度保持比例。

总结段落的`clear: both`让它在需要时向下避开前面的浮动，而不是取消图片的浮动。外层`flow-root`使图文容器包含内部浮动，后续其他内容从整个图文区域之后继续排列。

图4-14 活动介绍案例：三组规则共同完成图文环绕

三组规则分别回答图片怎样浮动、总结从哪里开始、父容器怎样包含内容。增加或删减介绍文字、缩小容器宽度后，仍应检查图片比例、文字环绕和后续内容的位置是否符合预期。

3 定位与位置参照

3.1 为什么需要定位

正常流通过前后关系组织正文，浮动通过环绕关系组织图文。页面还可能需要把标签放到卡片一角，把一个入口固定在窗口边缘，或者让标题滚动到某个位置后保持可见。这些需求强调的是元素与某个参照区域之间的位置。

定位通过`position`选择位置计算方式，再结合`top`、`right`、`bottom`、`left`等属性描述偏移或边界关系。使用定位时，要同时确定两件事：位置依据谁计算，以及正常流是否还为这个盒子保留空间。

前一个问题决定盒子显示在哪里，后一个问题决定其他内容是否会给它让位。单独知道一个`top`数值，并不足以解释布局结果。

图4-15 为什么需要定位：描述元素与参照区域的位置

3.2 position属性与定位参照

`position`用于选择元素的定位方式。元素默认采用`static`，按照所属布局的通常规则排列。定位方式及其占位特征可以概括如下。

理解定位时，要同时回答两个问题：**位置依据谁计算，正常流是否还为它保留空间。** 前一个问题决定元素显示在哪里，后一个问题决定其他内容是否会给它让位。

POSITION的值	正常流中的空间	基本位置关系
`static`	正常参与	按所属布局的通常规则排列
`relative`	保留原占位	相对原本位置偏移
`absolute`	不保留正常流占位	根据绝对定位包含块计算位置
`fixed`	不保留正常流占位	在常见简单页面中根据视口定位
`sticky`	保留流中空间	随滚动在指定边界处调整位置

图4-16 position决定位置计算方式，也决定占位关系

计算定位位置所用的参照区域称为包含块。它并不总是直接父元素，具体应结合定位方式及祖先规则判断。`top`等属性也不会对默认静态定位元素产生这里的偏移作用。

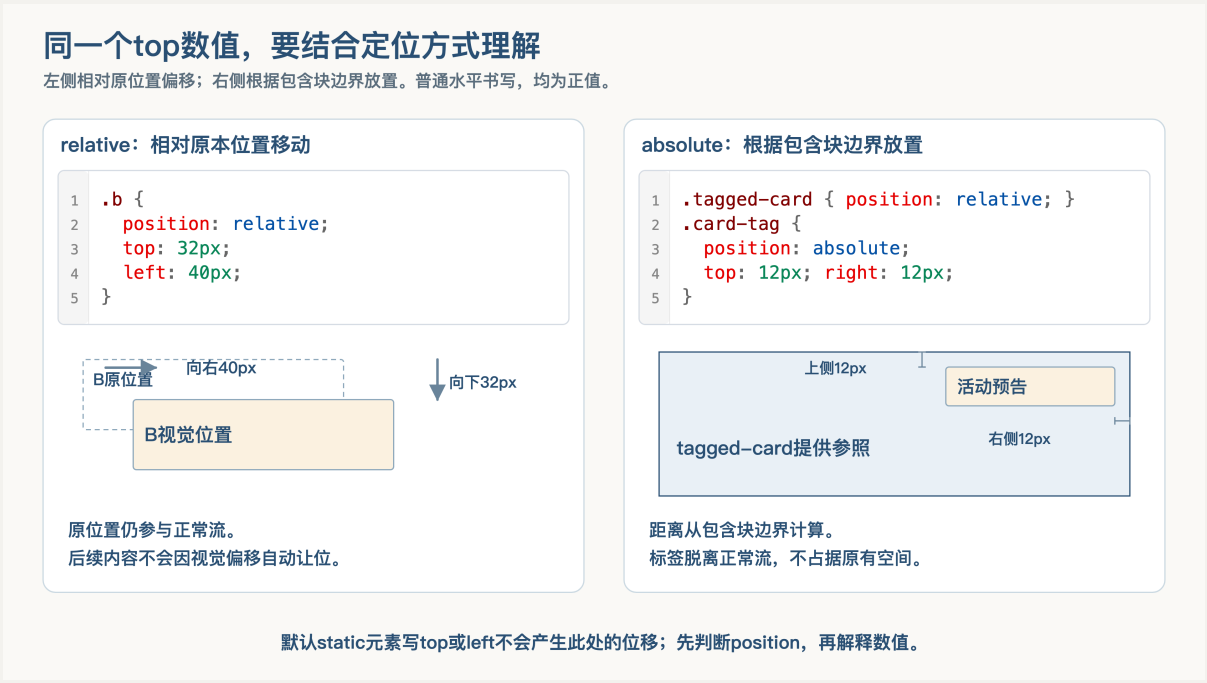

图4-17 同一个top数值，要结合定位方式理解

3.3 相对定位relative

相对定位先保留盒子按正常流取得的位置，再相对这个位置产生视觉偏移。它不会因为移动而重新安排后续盒子。

下面三个段落分别标为A、B、C。为了只比较位置，先去掉默认段落外边距：

```
<div class="position-row">
  <p>A 第一段内容</p>
  <p class="b">B 第二段内容</p>
  <p>C 第三段内容</p>
</div>
```

```
.position-row p {
  margin: 0;
  padding: 12px;
}

.b {
  position: relative;
  top: 32px;
  left: 40px;
}
```

浏览器先按照正常流为三个段落安排位置，再把B相对原来的位置向下移32px、向右移40px。这里正的`top`使它远离原位置的上侧，正的`left`使它远离原位置的左侧。

B移动后，正常流仍保留它原来的占位。C继续排在这个原占位之后，不会因为B视觉上向下移了32px，就自动再向下避让32px。于是B可能覆盖C的一部分，B原来的位置也可能露出空白。

图4-18 relative移动的是视觉位置，原占位仍然保留

这与给B增加`margin-top`不同。外边距参与正常布局，通常会影响其他盒子的排列；相对定位的视觉偏移发生在正常位置确定之后，不重新安排后面的段落。

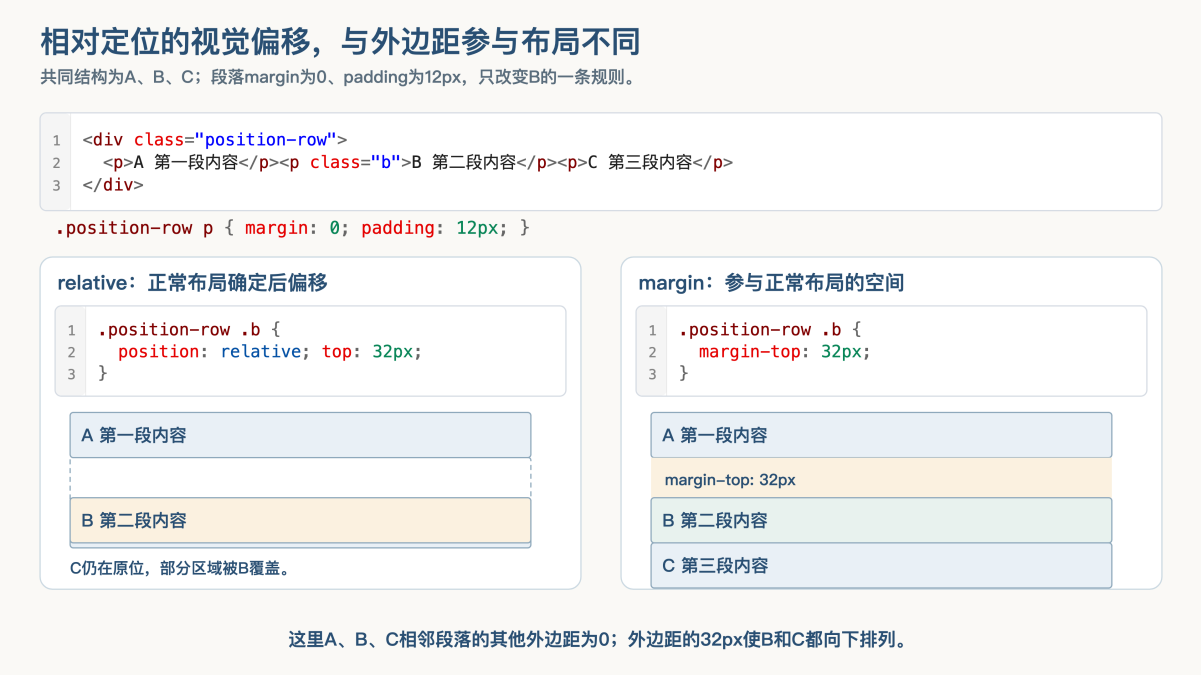

图4-19 相对定位的视觉偏移，与外边距参与布局不同

相对定位也可以不写任何偏移：

```
.tagged-card {
  position: relative;
}
```

卡片仍在原位置，但已经可以为内部绝对定位的标签提供参照。绝对定位的角标可以使用这种组合。

3.4 绝对定位absolute

绝对定位让盒子脱离正常流，不再保留原来的流中占位。其他内容不会因为这个盒子存在，就自动为它预留一行或一块空白。

绝对定位的位置依据它的包含块计算。在没有变换等特殊规则的常见页面中，可以沿祖先查找最近的非`static`定位元素；这个祖先为它提供参照。若没有找到相应祖先，就继续依据初始包含块等规则处理。因此，HTML直接父元素不一定就是实际参照。

常见组合是让容器采用相对定位，让内部标签采用绝对定位：

```css
.tagged-card {
  position: relative;
}

.card-tag {
  position: absolute;
  top: 12px;
  right: 12px;
}
```

容器没有偏移，仍保持原本位置，同时为标签建立包含块。在这个普通块容器例子中，包含块采用容器的内边距边缘；标签上边和右边分别离参照边界12px。

图4-20 absolute脱离正常流，后续内容不为它留位

如果外层`.card-stage`也采用`position: relative`，但把卡片改为`static`，标签就可能改用外层作为参照。恢复卡片的`relative`，标签又回到卡片一角。比较时，标签的偏移数值相同，变化来自哪一层建立包含块。

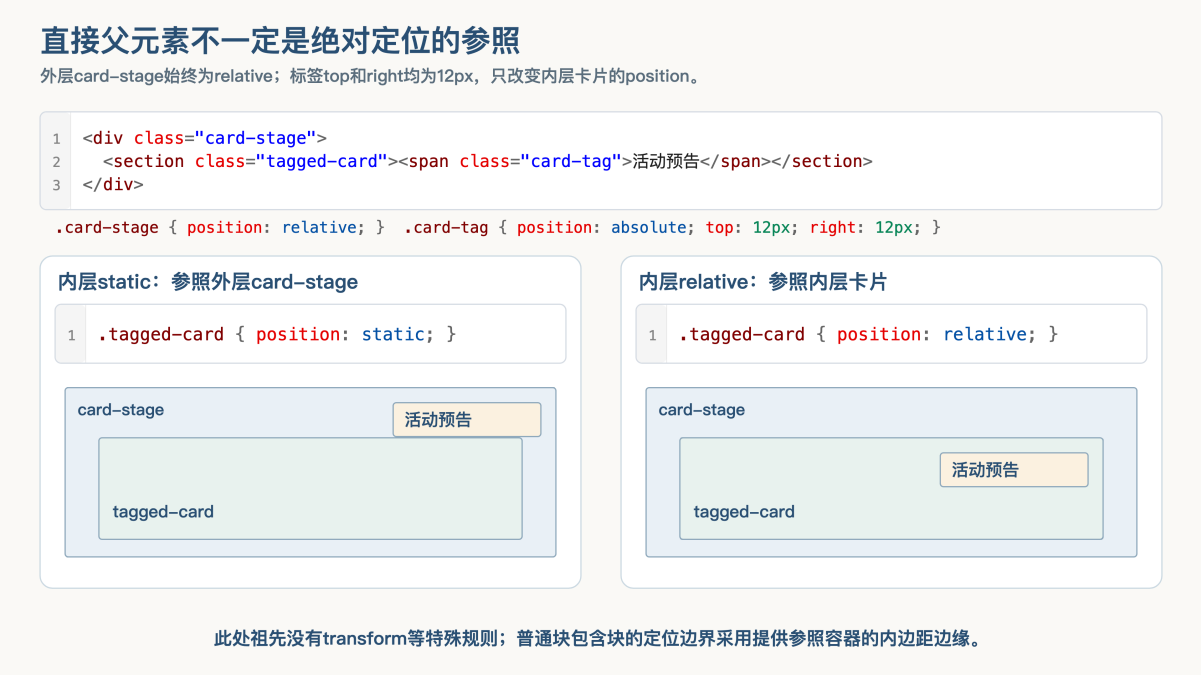

图4-21 直接父元素不一定是绝对定位的参照

绝对定位还需要处理空间预留。标签脱离正常流以后，正文不会主动避让它，需要通过容器留白或其他布局安排防止遮挡。第3.8节将把这些规则组合成完整卡片。

3.5 固定定位fixed

固定定位在常见简单页面中依据视口定位，盒子不保留正常流占位，页面滚动时仍停留在相应视口位置。祖先的变换等特殊规则可能改变包含块，扩展情况见附录B。

活动页面较长时，可以在浏览器窗口右下角保留一个“返回页首”入口。HTML仍然使用页内链接：

```
<header id="top">校园读书交流会</header>
<!-- 页面中的其他内容 -->
<a class="back-to-top" href="#top">返回页首</a>
```

```
.back-to-top {
  position: fixed;
  right: 16px;
  bottom: 16px;
  padding: 8px 12px;
  color: #ffffff;
  background-color: #205493;
}
```

在这个祖先没有设置变换等特殊规则的页面中，`fixed`让链接以视口为定位参照。视口是浏览器用于显示页面的区域，页面内容可以在其中滚动。`right`和`bottom`分别让入口离视口右侧、底部16px。

滚动页面时，正文的位置不断变化，链接却仍停留在视口右下角。它不需要跟随正文重新占一行，也不会自动为底部内容腾出位置。因此，如果最后一个表单控件恰好出现在右下角，就可能被这个入口遮住。

图4-22 fixed：正文滚动，入口仍在视口右下角

固定入口需要同时检查位置和遮挡。独立实验页在底部预留了空间，活动首页则保留普通流中的返回链接。链接能跳回页首来自`href="#top"`与对应ID；`position: fixed`只决定入口显示在哪里。

固定定位在具有特定祖先规则时可能采用其他包含块，相关情况见附录B。

3.6 粘性定位sticky

粘性定位保留盒子的流中空间，并在滚动过程中，根据相关滚动区域的边界调整位置。它同时受到自身包含块范围的约束。

有些内容只需要在某个区域内滚动时保持标题可见。例如，一个较长的阅读提示列表，希望标题到达区域顶部后暂时停住：

```
<section class="sticky-example" tabindex="0"
         role="region" aria-label="可滚动的阅读提示">
  <h3 class="sticky-heading">活动阅读提示</h3>
  <p>带一本你愿意分享的书。</p>
  <p>选择一段印象深刻的文字。</p>
  <p>提出一个希望讨论的问题。</p>
  <!-- 其余提示继续向下排列 -->
</section>
```

```
.sticky-example {
  height: 240px;
  overflow: auto;
  border: 1px solid #64748b;
}

.sticky-heading {
  position: sticky;
  top: 0;
  margin: 0;
  padding: 8px 12px;
  background-color: #e0edff;
}
```

父区域的高度限定为240px，内容超过可用高度时，`overflow: auto`让它能够内部滚动。这里的固定高度是为了建立滚动区域，不是为了容纳整页正文。`tabindex="0"`让该区域可以通过Tab取得焦点，便于键盘滚动。

标题最初仍按正常位置排列。内部内容向上滚动、标题将要越过上边界时，`top: 0`使它在允许范围内保持靠近该滚动区域顶部。标题的流中空间仍然保留，它也受自身包含块的边界限制，不会无限跟随整个页面。

图4-23 sticky：标题留在滚动区域的边界附近

与上一节的固定入口相比，两者的参照与占位都不同：固定入口在简单页面中依据视口定位并脱离正常流；这里的粘性标题依据内部滚动区域的边界调整位置，仍保留流中空间。

图4-24 sticky与fixed：滚动的对象不同，参照也不同

如果没有出现预期效果，应先确定实际滚动的是页面还是内部区域，再检查`top`是否设置，以及可滚动内容和容器空间是否足够。祖先的`overflow`会影响滚动关系，不能把所有`sticky`都理解成“滚到顶部后固定在窗口”。

3.7 z-index与覆盖顺序

定位盒子可以相互重叠。两个盒子的位置已经确定后，还需要判断重叠部分先后绘制的关系：

```
<div class="layer-demo">
  <div class="layer-a">A</div>
  <div class="layer-b">B</div>
</div>
```

```
.layer-demo {
  position: relative;
  height: 140px;
}

.layer-a,
.layer-b {
  position: absolute;
  width: 160px;
  padding: 16px;
  border: 1px solid #64748b;
}

.layer-a {
  top: 12px;
  left: 12px;
  z-index: 1;
  background-color: #e0edff;
}

.layer-b {
  top: 40px;
  left: 80px;
  z-index: 2;
  background-color: #ffedd5;
}
```

A和B都依据同一个父容器定位，坐标让它们的一部分区域相交。在这个简单例子中，两个定位盒子处于同一层叠上下文，B的`z-index`为2，大于A的1，因此B覆盖A的相交部分。

若交换两个数值，A就显示在上面。盒子的坐标和尺寸都没有改变，变化的是覆盖顺序。由此可以区分位置问题与层级问题：元素跑到错误角落时，应先查定位参照；元素位置正确却被挡住时，才继续查覆盖关系。

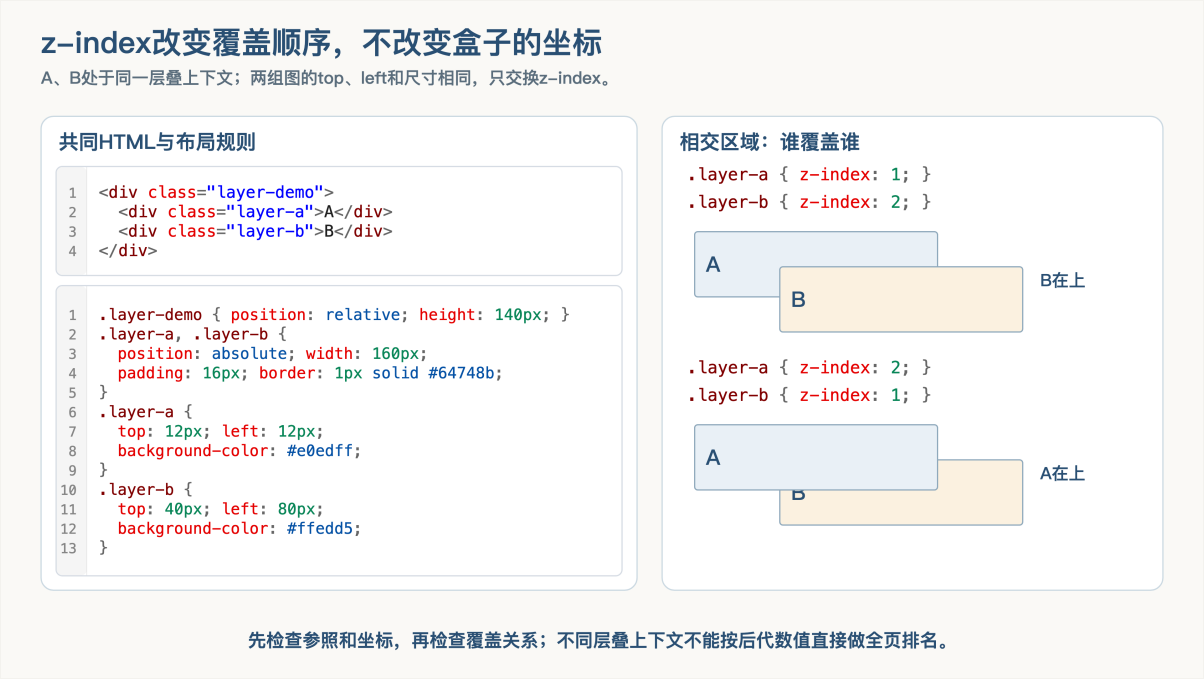

图4-25 z-index改变覆盖顺序，不改变盒子的坐标

层叠上下文可以理解为一组内容内部比较层级的范围。不同上下文在外部还会作为整体参加比较，所以`z-index`并不是全页面所有后代共用的一张排名表。相关扩展见附录B。

3.8 案例 在活动卡片右上角放置角标

活动介绍的“活动预告”是一个短标签，适合相对卡片定位；活动标题与正文则继续留在正常流中。将两层关系写在同一张卡片内：

```
<section class="activity-card tagged-card">
  <span class="card-tag">活动预告</span>
  <h2>校园读书交流会</h2>
  <p>带一本喜欢的书，分享一次阅读体验。</p>
</section>
```

图4-26 活动角标案例：先明确三层内容的关系

```css
.tagged-card {
  position: relative;
  padding: 64px 24px 24px;
  border: 1px solid #cbd5e1;
}

.card-tag {
  position: absolute;
  top: 12px;
  right: 12px;
  padding: 4px 10px;
  background-color: #205493;
  color: #ffffff;
}
```

卡片没有设置偏移，仍按原有方式参与页面排列，但为角标建立了定位参照。角标脱离正常流，离参照区域上边和右边各12px。顶部64px内边距为它预留空间，标题与正文才不会紧贴角标。

图4-27 活动角标案例：参照、坐标与留白共同起作用

若删除卡片的`relative`，角标会继续依据祖先或初始包含块等规则定位；若只保留定位而不预留空间，角标又可能覆盖标题。这分别对应定位参照与正常流占位两个问题。

图4-28 角标跑错位置与正文被遮挡，要分别检查

标签较长时应重新检查空间，必要时将它放回正常流。正文卡片不宜都用一组绝对坐标排满页面：上方内容变高时，下方固定坐标不会自动跟随，容易重叠。卡片之间的排列应继续由正常流或布局容器组织。

4 Flexbox弹性布局

4.1 为什么需要Flexbox及其基本模型

前面已经学习了浮动与定位。使用这些规则，可以让几个区域左右排列，也可以把链接放到容器一侧。页面布局的要求继续增加以后，还会遇到另一类问题：一组长短不同的内容怎样共同对齐，空间怎样分配，文字变长或容器变窄以后怎样继续保持合理排列？

先看三张书目卡片。给每张卡片设置宽度并向左浮动，可以把它们排成一行，再用外边距留下间隔：

```
.book-list {
  display: flow-root;
  width: 600px;
}

.book-card {
  float: left;
  width: 180px;
  margin: 0 16px 0 0;
}

.book-card:last-child {
  margin-right: 0;
}
```

父容器使用`flow-root`，已经能够包含内部浮动。但三张卡片的高度仍各自随内容增长：短介绍的卡片较矮，长介绍的卡片较高。浮动本身没有提供让这一行卡片自动等高的规则。若继续要求它们等高、内部链接贴近底部，就还要增加相应的处理；简单写死同一个高度，又可能在文字增加时容纳不下内容。

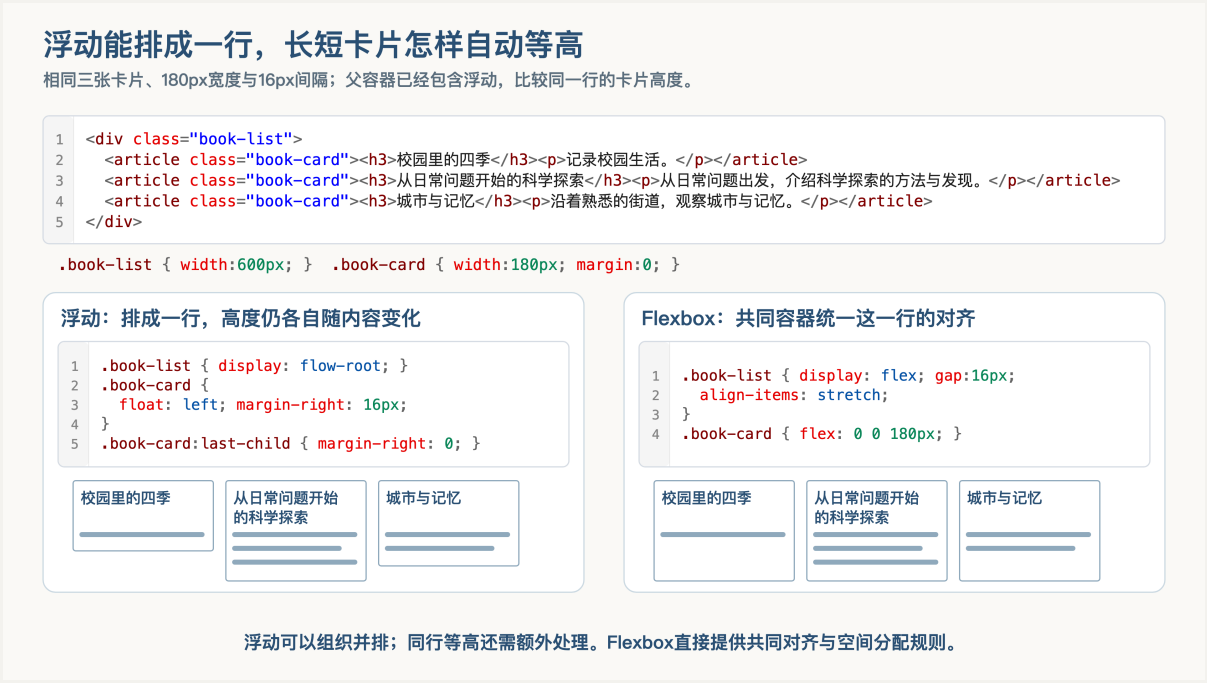

图4-29 浮动能排成一行，长短卡片怎样自动等高

再看活动介绍底部的操作区。左边是一段说明，右边是两个链接，可以先把链接组绝对定位到右上方：

```
.action-bar {
  position: relative;
}

.action-links {
  position: absolute;
  top: 0;
  right: 0;
}
```

链接组虽然到了右侧，却脱离了正常流。说明文字不会主动为它留出宽度，文字变长时可能排到链接下方。可以通过预留空间等方式避免遮挡，但链接组的宽度、说明长度或容器宽度变化以后，还需要继续维护这些条件。如果再要求两组内容竖直居中、空间不足时自然换行，单独指定位置会更麻烦。

图4-30 定位能把链接放到右侧，正文变长后怎样避让

这两种需求都需要把一组元素放在共同容器中，统一处理尺寸、对齐和剩余空间，而不是分别安排每个盒子的宽度、外边距或坐标。**Flexbox**，即弹性盒布局，为这类排列提供了专门的布局方式。

Flexbox以主轴组织项目，通过共同容器管理一组直接子项目。项目可以在允许范围内增长或收缩，也可以根据规则换行。例如，同一行中高度为自动的卡片可以通过交叉轴拉伸取得相同高度；操作区中的说明和链接组可以共同参与空间分配，空间不足时再换行。相应规则将在后续小节逐步说明。

浮动仍用于图文环绕，定位仍用于角标、固定入口等局部位置关系。选择布局方式，要看当前需要处理的是文字环绕、局部坐标，还是一组项目之间的尺寸与排列关系。

理解Flexbox首先需要区分两个对象：弹性容器负责方向、换行、对齐与间距；弹性项目负责自身的基础尺寸、伸缩和局部对齐。容器内部的主轴与交叉轴则提供方向依据。

```
.list {
  display: flex;
  gap: 16px;
}
```

`display: flex`建立弹性容器，正常流中的直接子元素成为弹性项目。它改变容器内部的排列规则，不要求容器自己脱离外部正常布局。

HTML提供内容及父子关系，CSS提供尺寸和排列规则，浏览器结合空间计算最终位置。需要共同排列哪些项目，必须先从HTML结构确定；更深层的后代不会自动越过父元素成为这个容器的项目。

图4-31 display:flex写在容器上，直接子元素成为项目

容器建立以后，子项目不需要再逐个写`display: flex`才参与排列。若要控制项目自己的尺寸和伸缩，可以把`flex`属性写在子项目上。下面用A、B、C三个简单项目说明父容器与子项目的写法：

```
<div class="list">
  <div class="item">A</div>
  <div class="item">B</div>
  <div class="item">C</div>
</div>
```

```
.list {
  display: flex;
  width: 400px;
  gap: 16px;
}

.item {
  flex: 0 0 100px;
}
```

`flex`是一种简写，这里的三个值依次对应`flex-grow`、`flex-shrink`和`flex-basis`。第一个`0`表示项目不分得剩余空间而增长；第二个`0`表示项目不因空间不足而收缩；`100px`表示项目从100px的基础尺寸参与布局。前两个数值表达增长与收缩的分配规则，第三个值表达基础尺寸，不能把它们当作三组宽度。

本例采用默认的横向排列，项目没有额外内边距、边框、外边距或其他尺寸限制。因为既不增长也不收缩，三个项目的最终宽度都是100px。容器宽400px，项目共占300px，两个间隔共占32px，末端还剩68px；这些空间不会自动把项目拉宽。如果把容器缩窄到不足332px，由于第二个值是`0`，项目不会自动缩小，默认不换行时就会超出容器。

图4-32 flex写在子项目上：先读懂三个值

4.2 主轴 交叉轴与排列方向

Flexbox使用两条相互垂直的轴描述方向。项目主要沿着主轴排列，与主轴垂直的方向称为交叉轴。`flex-direction`决定主轴采用哪个方向：

```
.site-nav {
  display: flex;
  flex-direction: row;
}
```

本章示例采用水平书写、从左到右的文字方向。在这个条件下，`row`使主轴从左向右，三个链接依次横向排列；交叉轴从上向下。

图4-33 先认识Flex容器、弹性项目与两条轴

只把方向改成`column`：

```
.site-nav {
  display: flex;
  flex-direction: column;
}
```

主轴就变成从上向下，三个链接沿竖直方向依次排列；交叉轴变为水平方向。链接仍然是弹性项目，变化的是组织它们的方向。

FLEX-DIRECTION	本章条件下的主轴	本章条件下的交叉轴
`row`	从左向右	从上向下
`column`	从上向下	从左向右

后面介绍的`justify-content`始终依据主轴，`align-items`始终依据交叉轴。方向改变以后，它们对应的水平、竖直方向也会改变。因此，先确定轴，再解释对齐，比把某个属性记成“水平居中”更准确。

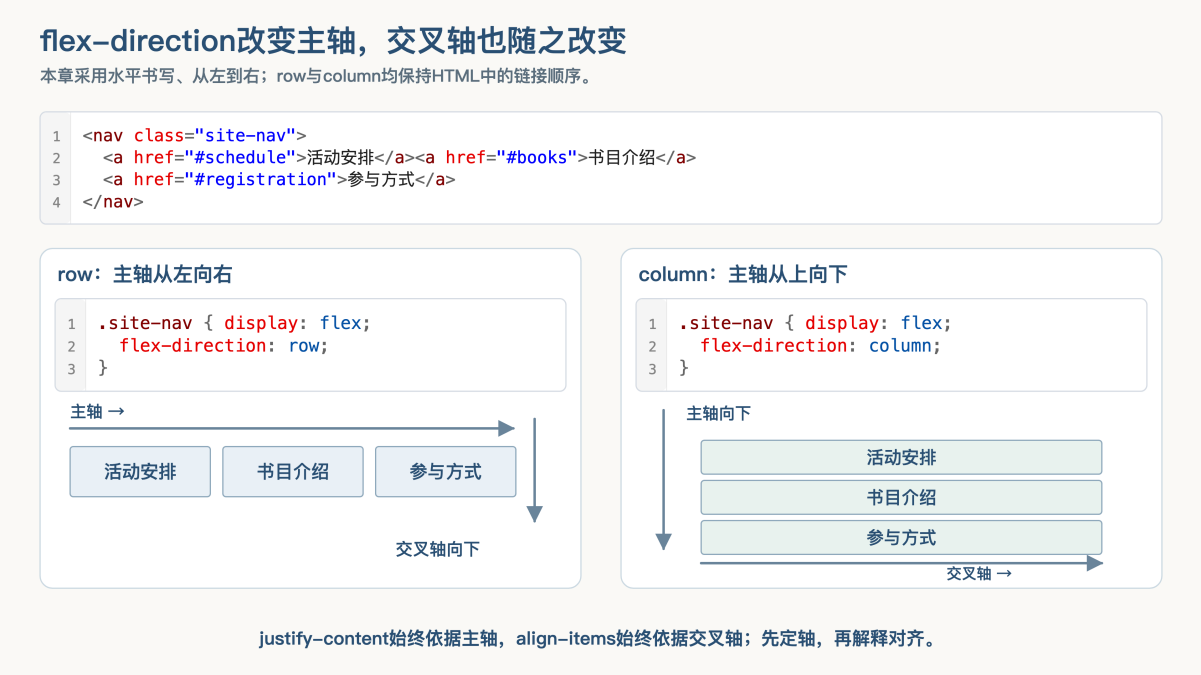

图4-34 flex-direction改变主轴，交叉轴也随之改变

`row-reverse`和`column-reverse`还可以反转视觉排列方向，但不会改写HTML内容顺序。页面的阅读和键盘操作需要连贯，本章案例保持源代码顺序与主要视觉顺序一致。

4.3 Flex的基本概念和用法

前两节已经说明，Flexbox通过共同容器组织直接子项目，并用主轴、交叉轴描述排列方向。实际使用时，可以先看四件事：项目怎样对齐，项目之间留多少距离，空间不足时是否换行，以及项目宽度是否允许随空间调整。

让项目按需要对齐。

先用同一组活动信息观察主轴与交叉轴对齐。地点有两行，其余项目只有一行：

```
<div class="info-row">
  <div>周五下午</div>
  <div>图书馆<br>交流空间</div>
  <div>欢迎参加</div>
</div>
```

`justify-content`控制主轴上的对齐。下面采用默认的`row`方向，给容器设置600px宽度，让三个项目按内容和内边距取得宽度，容器中保留相应的剩余空间：

```css
.info-row {
  display: flex;
  width: 600px;
  max-width: 100%;
  justify-content: flex-start;
}

.info-row > div {
  padding: 8px 16px;
}
```

`flex-start`让项目从主轴起点依次排列，剩余空间留在末端。只把它改为`center`，项目作为一组居中，剩余空间分到两端；改为`space-between`，首尾项目靠近两端，剩余空间分配到相邻项目之间。这组对照不设置项目增长、外边距或`gap`，只改变`justify-content`，便于看清主轴对齐的作用。

图4-35 justify-content决定主轴上剩余空间的分配

`justify-content`始终依据主轴，方向改变后，需要重新判断对齐方向。如果项目已经占满可用空间，改成`center`或`space-between`也不一定出现明显的位置变化。

`align-items`控制交叉轴上的对齐。保持前面的HTML、容器宽度和项目内边距，在容器上增加下面的规则：

```
.info-row {
  align-items: center;
}
```

默认row方向下，交叉轴是竖直方向。align-items: center让较矮项目在同一行中竖直居中；改为flex-start或flex-end，则分别靠近这一行的上侧或下侧。这里对齐的是项目盒子，不会自动改变盒子内部文字的水平对齐。

图4-36 align-items对齐同一弹性行内的项目盒子

卡片列表还常用align-items: stretch：在同一行中，高度为自动等条件满足时，较矮卡片可以被拉到这一行的高度。第4.4节将用长短书目卡片组合应用这个规则。

用gap表达项目间距。

gap写在容器上，统一设置相邻项目之间的间隔。下面让导航链接之间保留16px，同时让内容与容器边缘之间保留12px：

```
<nav class="site-nav">
  <a href="#schedule">活动安排</a>
  <a href="#books">书目介绍</a>
  <a href="#registration">参与方式</a>
</nav>
```

```css
.site-nav {
  display: flex;
  gap: 16px;
  padding: 12px;
}

.site-nav a {
  margin: 0;
}
```

`gap`用于项目之间，`padding`用于容器边缘的内部留白。若项目原来还有外边距，实际距离可能继续增加；修改AI生成的导航样式时，应同时检查项目自己的`margin`，避免多种间距来源叠加。

图4-37 gap在项目之间，padding在容器边缘

允许空间不足的项目换行。

Flex容器默认采用`nowrap`，项目留在同一条弹性行中，并可能尝试收缩。导航或卡片需要在空间不足时进入下一行，可以给容器设置`flex-wrap: wrap`。下面用一组信息项观察这一变化：

```html
<div class="wrap-row">
  <div>A：较长的活动说明……</div>
  <div>B：书目介绍……</div>
  <div>C：参与方式……</div>
</div>
```

```css
.wrap-row {
  display: flex;
  width: 600px;
  max-width: 100%;
  gap: 12px;
  flex-wrap: wrap;
}

.wrap-row > div {
  flex: 1 1 180px;
  min-width: 0;
}
```

保持上面的规则，把容器宽度从600px改为400px，可以观察第三个项目整体进入下一条弹性行。一个项目内部的句子变成多行，仍然可能只是文字换行；项目盒子整体移到另一条弹性行，才是项目换行。

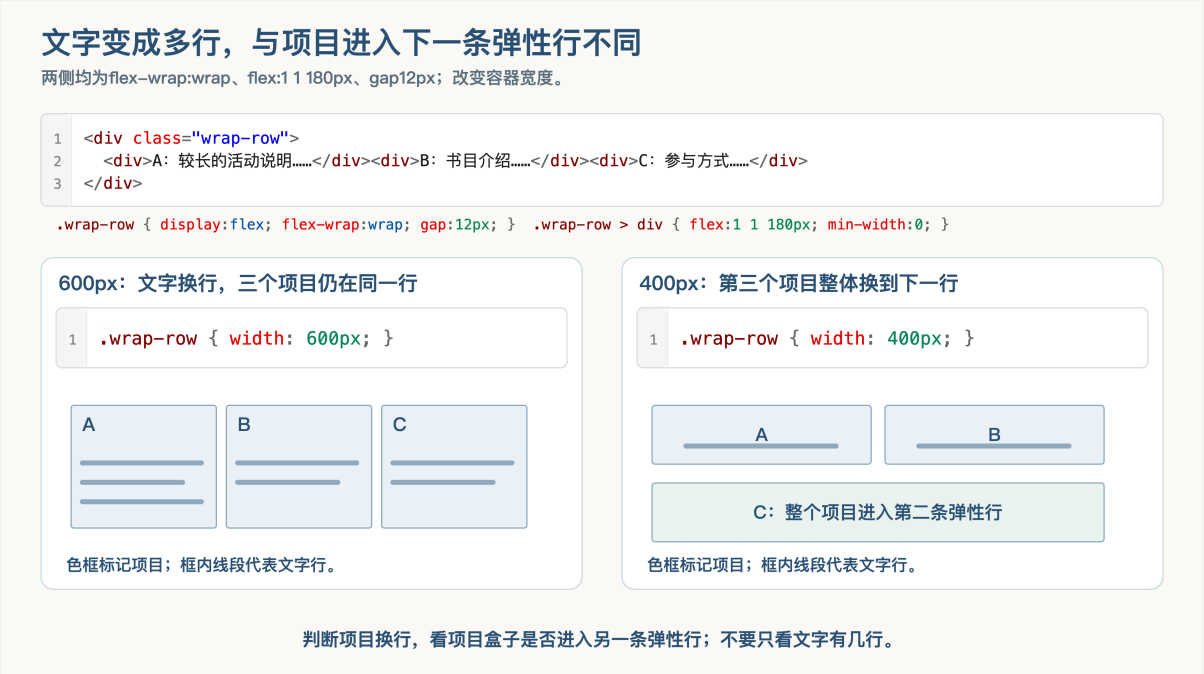

图4-38 文字变成多行，与项目进入下一条弹性行不同

理解宽度可以随空间调整。

项目上的`flex: 1 1 180px`依次表示允许增长、允许收缩，以及从180px基础尺寸参与排列。空间充足时，项目可以分得剩余空间；空间不足时，要结合基础尺寸、换行规则及尺寸限制决定怎样排列。180px不是对最终宽度的固定承诺。`min-width: 0`允许项目在需要时进一步缩小，长字符串或内部图片仍需检查各自的换行与尺寸规则。

为了单独观察增长与收缩，可以暂时把上例容器改为`flex-wrap: nowrap`，其他规则不变。在没有额外尺寸限制的条件下，容器宽600px时，三个项目可以分得剩余空间；改为400px时，它们仍留在同一行，并缩小以适应空间。两种情况下的基础尺寸都是180px，最终宽度却不同。

```
.wrap-row {
  flex-wrap: nowrap;
}
```

图4-39 基础尺寸不是固定宽度：项目可以增长，也可以收缩

保留原来的`wrap`时，空间不足还可能先触发换行，再在每条行中分别分配空间。最后一行只有一个项目时，它也可能因为允许增长而铺满这一行；Flexbox不会自动让所有行共享同一组列宽。

阅读或修改AI生成的Flex代码时，应能从容器找到这些规则，并用增加文字、缩窄空间或局部修改属性来验证解释。对齐的其他取值、精确尺寸计算与逐行分配的详细过程，放在附录E中查阅。

4.4 案例 排列操作区与长短书目卡片

布局经常需要分成两层。活动介绍底部，左边是一句说明，右边是一组链接；外层需要排列“说明”和“链接组”，链接组内部又需要排列两个链接。

```
<div class="action-bar">
  <p class="action-note">欢迎带着问题参与讨论。</p>
  <div class="action-links">
    <a href="#registration">查看参与方式</a>
    <a href="#schedule">阅读活动安排</a>
  </div>
</div>
```

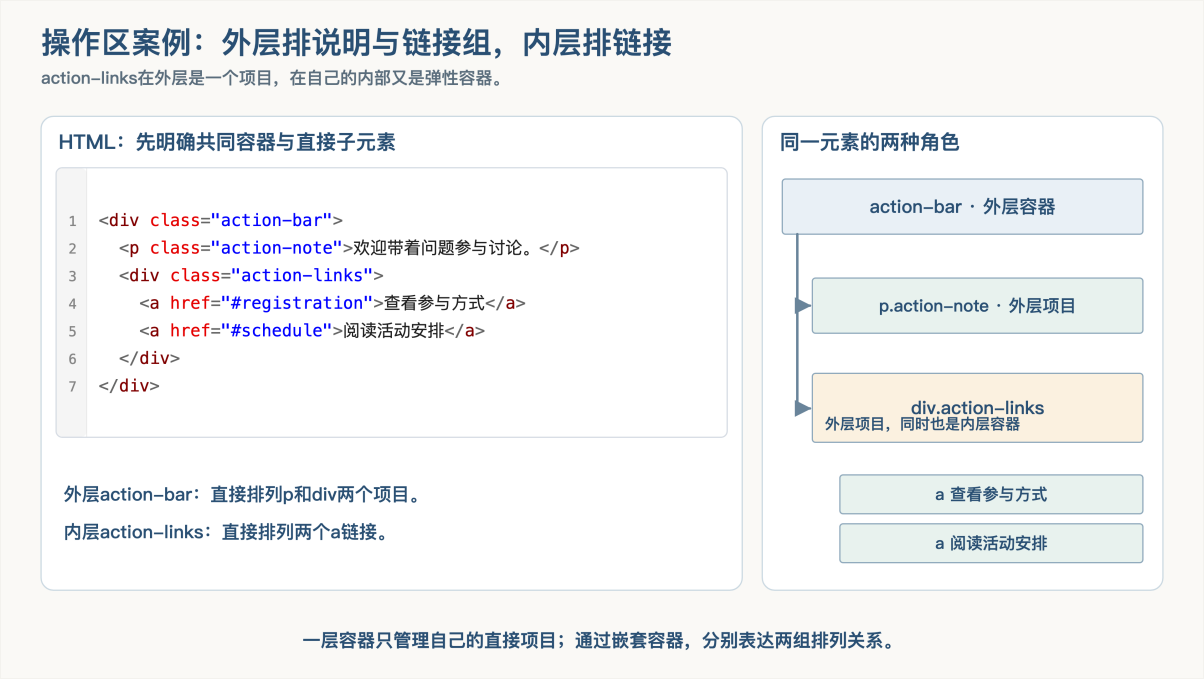

图4-40 操作区案例：外层排说明与链接组，内层排链接

```
.action-bar {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 12px;
}

.action-note {
  flex: 1 1 220px;
  margin: 0;
}
```

```css
.action-links {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}
```

外层`.action-bar`的直接子元素是段落与链接组。段落从220px基础尺寸参与分配，并允许增长；`.action-links`作为一个整体参加外层排列。它自己又是一个弹性容器，用8px间距组织内部链接。因此，同一个元素可以在外层是项目，在内层是容器。

图4-41 操作区案例：两层Flex各自负责排列与换行

三张长短不同的书目卡片也需要类似的分层处理。外层让卡片横向排列，内层让标题、介绍和链接从上到下排列：

```html
<div class="book-list">
  <article class="book-card">
    <h3>校园里的四季</h3>
    <p>从一条熟悉的小路开始，记录校园生活。</p>
    <a href="#registration">参与这本书的讨论</a>
  </article>
  <!-- 另外两张卡片结构相同，标题与介绍长度不同。 -->
</div>
```

图4-42 书目卡片案例：外层按行，内层按列

```css
.book-list {
  display: flex;
  flex-wrap: wrap;
  align-items: stretch;
  gap: 16px;
}

.book-card {
  display: flex;
  flex-direction: column;
  flex: 1 1 220px;
  min-width: 0;
}

.book-card > a {
  margin-top: auto;
  align-self: flex-start;
}
```

外层的主轴水平，`stretch`让同一行中高度为自动的卡片拉到这一行的高度。长介绍撑高所在行，短介绍所在的卡片也得到相同高度，但内部会剩下一块空间。

图4-43 书目卡片案例：同行等高，内层链接推到底部

内层卡片使用`column`，主轴变成竖直方向。链接的`margin-top: auto`吸收上方可分配的正剩余空间，把链接推到卡片底部。长卡片剩余空间少，短卡片剩余空间多，各自吸收以后，同一行的底部链接就能对齐。

`align-self`用于单独设置一个项目的交叉轴对齐。卡片交叉轴是水平方向，`align-self: flex-start`使链接靠起点，而不是横向拉伸为整张卡片宽。它只调整这个链接，不改变其他项目共同采用的`align-items`规则。

图4-44 卡片已经等高，为什么链接还需要margin-top:auto

这里的等高针对同一弹性行。换行后，不同行的卡片仍分别计算高度与空间，不会因此自动形成贯穿所有行的等高网格。

4.5 使用开发者工具检查弹性布局

布局是否合理，需要结合页面结果和实际生效的规则判断。在页面中右键检查导航或卡片，先找到共同容器，再看它的直接子元素。确认`display: flex`写在需要组织项目的那一层，而不是误写到某个内部文字元素上。

选中容器后，在Styles中检查方向、对齐、`gap`与换行声明。浏览器提供的Flex布局标记或叠加显示，可以帮助辨认弹性行与间隔；选中一个项目后，再通过Computed和盒模型查看它的实际尺寸与边距。

检查对象	要能回答的问题
容器与直接子元素	当前是哪一层在组织哪些项目？
方向与对齐	主轴指向哪里，当前属性沿哪个方向起作用？
间距与尺寸	留白来自gap、padding还是margin，项目是否允许调整宽度？
换行与内容	是项目进入下一行，还是项目内部文字换行？

可以在布局实验页临时开关`flex-wrap`，或在活动页中增加一段说明、缩窄容器，再观察布局怎样变化。若修改了一个对齐属性却看不出效果，应检查轴方向、可分配空间和其他生效规则。若内容被遮挡或挤出边界，要检查尺寸限制与内容本身，不能仅用裁切掩盖问题。

开发者工具中的修改通常是临时的。确认结果符合预期后，应将修改保存回CSS源文件，再刷新页面复核。附录E.7提供读取基础尺寸与最终宽度的进一步例子。

5 综合案例 校园活动页面布局

5.1 沿用内容并增加局部布局

案例继续使用校园读书交流会专题页，包含页头、导航、活动介绍、活动安排、交流书目、参与提醒与静态表单。活动时间、地点和书目均为虚构教学素材。

页面主体仍按HTML顺序上下排列，各区域保留CH03的背景、边框和留白。本章在这些基础上增加局部布局：介绍图片与文字形成环绕，短角标放在卡片右上角，导航和时间地点信息使用Flexbox，书目卡片内部通过列布局安排内容。

页面区域	结构与布局
页头导航	一个nav包含多个链接，父容器组织排列
活动介绍	卡片为角标定位，内部图文容器包含浮动
时间地点信息	三个信息项按可用宽度伸缩与换行
交流书目	外层排列卡片，卡片内部排列标题、介绍和链接
活动安排与参与方式	保留表格与表单原有语义，整体随正文排列

完整文件位于src/ch04，活动首页为index.html。固定定位、粘性定位和独立尺寸实验位于layout-lab.html，便于在简单结构中比较。

页面上的链接提供页内跳转或页面跳转。静态表单保留输入与原生校验结构，提交按钮保持禁用，当前页面不处理报名信息。

5.2 从CH03导航迁移到Flexbox

导航沿用CH03的链接内容，保留每个链接的颜色、边框和内边距，把组织整组链接的规则移到共同父容器：

```
.site-nav {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 8px 12px;
}

.site-nav a {
  margin: 0;
  padding: 6px 12px;
  border: 1px solid #ffffff;
  border-radius: 6px;
  color: #ffffff;
  text-decoration: none;
}
```

`nav`成为容器，链接成为项目。`justify-content: center`让同一行的链接组居中，`gap`统一提供行间与项目间距离。链接上原有的`margin: 4px`重设为零，避免它与新的间距叠加。

此时，链接不再依靠`inline-block`组成这一组排列。项目数量、文字长度或容器宽度变化后，父容器会按同一组Flex规则重新安排它们。启用`wrap`后，放不下的链接进入下一行，每条行再分别居中。

完整案例增加了“交流书目”和“布局实验”入口，仍然采用相同规则。实际换行位置由链接文字、内边距、边框、项目间距和容器内容宽度共同决定；增加一个入口，就可能让换行更早发生。

5.3 让角标与图文环绕各司其职

活动介绍卡片同时包含角标、标题和图文内容。这几部分的结构如下：

```
<section id="introduction" class="activity-card tagged-card">
  <span class="card-tag">活动预告</span>
  <h2>校园读书交流会</h2>
  <div class="reading-intro">
    <img class="reading-cover" src="images/activity.svg"
         alt="展开的书与交流气泡的插画" width="800" height="300">
    <p>带一本喜欢的书，分享一次阅读体验。</p>
    <p>从一个问题开始，让不同的想法在交流中相遇。</p>
  </div>
</section>
```

这张卡片同时具有`activity-card`与`tagged-card`两个类。前者提供已有的边框和24px内边距，后者补充定位参照并覆盖顶部内边距：

```
.tagged-card {
  position: relative;
  padding-top: 64px;
}

.card-tag {
  position: absolute;
  top: 12px;
  right: 12px;
}

.reading-intro {
  display: flow-root;
}
```

```css
.reading-cover {
  float: left;
  width: 160px;
  max-width: 45%;
  height: auto;
  margin: 0 16px 8px 0;
}
```

`.tagged-card`为角标提供参照，`padding-top`给它留出空间。角标不在正常流里，正文标题仍按自己的正常位置排列。这里使用短标签，64px顶部内边距足够形成间隔，实际文字放大后仍需核对。

图文部分是另一层关系。图片在`.reading-intro`内部浮动，介绍文字沿图片环绕；`flow-root`让这个区域包含内部浮动，后面的时间地点信息从整个介绍区域下方继续排列。图片宽度限制为父容器的45%以内，较窄时可以继续缩小，`height: auto`保持比例。

角标的参照由卡片提供，浮动的边界由图文容器组织。两者共同出现在同一张卡片里，各自作用于不同层级，不需要互相替代。

5.4 用信息项与书目卡片观察尺寸和换行

活动时间、地点和参与方式属于三组简短信息，可以用定义列表表达：

```html
<dl class="activity-facts">
  <div class="fact-item"><dt>时间</dt><dd>周五 14:00–15:00</dd></div>
  <div class="fact-item"><dt>地点</dt><dd>图书馆交流空间</dd></div>
  <div class="fact-item"><dt>参与方式</dt><dd>带一本喜欢的书</dd></div>
</dl>
```

每个`div`将一组名称`dt`与说明`dd`组织为一个项目，外层列表负责排列：

```css
.activity-facts {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
}

.fact-item {
  flex: 1 1 180px;
  min-width: 0;
  padding: 12px;
  border: 1px solid #cbd5e1;
}
```

完整样式采用统一的`border-box`。这里的180px基础尺寸包含项目的内边距与边框，分析一行能否放下时，还要加上项目之间的`gap`。空间充足时，三项可在同一行增长；不足时，项目进入下一行，各行再独立分配。

交流书目使用同样的外层思路，但内部内容更长。三张卡片分别介绍“校园里的四季”“从日常问题开始的科学探索”和“城市与记忆”，标题与介绍长度不同。卡片结构仍然一致：

```html
<div class="book-list">
  <article class="book-card">
    <p class="book-category">文学 · 示例书目</p>
    <h3>校园里的四季</h3>
    <p>从一条熟悉的小路开始，记录校园生活。</p>
    <a class="action-link" href="#registration">参与这本书的讨论</a>
  </article>
  <article class="book-card">
    <p class="book-category">科普 · 示例书目</p>
    <h3>从日常问题开始的科学探索</h3>
    <p>为什么天空有不同的颜色？为什么雨后会出现彩虹？从身边的问题出发，分享你找到的解释，以及还没有解决的疑问。</p>
    <a class="action-link" href="#registration">参与这本书的讨论</a>
  </article>
  <article class="book-card">
    <p class="book-category">人文 · 示例书目</p>
    <h3>城市与记忆</h3>
    <p>一条街道、一座建筑，怎样承载一座城市的共同记忆？带着你熟悉的地方参与交流。</p>
    <a class="action-link" href="#registration">参与这本书的讨论</a>
  </article>
```

```
</div>
```

```
.book-list {
  display: flex;
  flex-wrap: wrap;
  align-items: stretch;
  gap: 16px;
}

.book-card {
  display: flex;
  flex-direction: column;
  flex: 1 1 220px;
  min-width: 0;
  padding: 16px;
  border: 1px solid #cbd5e1;
}

.book-card > .action-link {
  margin-top: auto;
  align-self: flex-start;
}
```

外层从220px基础尺寸组织卡片，同一行采用`stretch`。内层的列布局让自动上外边距吸收正文下面的剩余空间，把操作链接推到底部。标题、介绍长短不同，也能在同一行取得整齐的底部位置。

图中的实际页面使用这三张书目卡片。宽视口下，三张卡片位于同一行，行高由内容较多的卡片参与决定；窄视口下，三张卡片分成三行，每行只有一个项目，各自按内容取得所需高度。

图4-45 宽容器中的书目卡片与底部对齐

图4-46 窄容器中的书目卡片自然换行

这时还没有使用媒体查询，变化来自流动宽度与Flex换行。若需要明确限制列数、保持跨行列宽，或同时调整页面主区域，CH05将继续用同一组书目学习Grid与响应式布局。

5.5 保留原有语义与操作能力

布局调整之后，页面仍需保持原有内容关系。活动安排中的时间与内容由`table`、`th`和`td`表达；参与方式中的标签和控件通过`for`与`id`关联。这些关系不会因为外层采用Flexbox就自动改变，也不能用视觉上的相邻代替。

```
<div class="form-field">
  <label for="nickname">练习昵称</label>
  <input id="nickname" name="nickname" type="text" required>
</div>
```

这里标签与控件的关联来自属性，而不是“恰好排在输入框上面”。同样，导航和操作入口继续使用链接，表单提交继续使用相应按钮，CSS只安排它们的外观与空间。

键盘焦点也需要在新布局中清楚可见。完整案例沿用CH03的焦点样式：

```
a:focus-visible,
input:focus-visible,
select:focus-visible,
textarea:focus-visible {
  outline: 3px solid #92400e;
  outline-offset: 3px;
}
```

按Tab键依次移动时，应能辨认当前链接或控件，且它们不被角标或固定入口遮住。视觉顺序也应与HTML阅读顺序基本一致；仅用`order`或反向排列移动外观，并不会同步改写源代码顺序。

活动页的表单仍是静态练习，提交按钮禁用。表单排得整齐与报名业务能够接收、保存数据，是不同层面的完成情况。

5.6 布局破案练习

假设导航原本能够居中排列，修改后却失去原来的间距和换行。检查样式表，发现`display: flex`被移到了链接上：

```
.site-nav a {
  display: flex;
}
```

这条规则让每个链接成为自己内部内容的弹性容器，却没有让`nav`组织这组链接。即使链接文字恰好仍排在一行，容器层级已经与原设计不同。应先选中`nav`，确认共同容器采用`display: flex`，再检查其方向、间距和换行声明。

类似现象还可能来自其他原因，可以逐项判断：

局部修改	对页面的影响	检查与修复依据
删除卡片的`position: relative`	角标可能改用其他祖先或初始包含块定位	找到实际参照，恢复卡片定位
保留链接原来的`margin: 4px`后增加gap	项目距离比预期大	核对外边距与gap的共同作用
把信息项改为`flex: 0 0 400px`	窄容器中项目不能缩小，可能溢出	核对基础尺寸、收缩规则和容器宽度
关闭书目链接的`margin-top: auto`	短介绍卡片的链接可能不再贴底	核对内层列布局及剩余空间
把短角标改成长段文字	预留顶部空间可能不足，遮住正文	检查标签高度，必要时改用正常流

先描述具体变化，再提出原因，使用工具检查后做小范围修复。给外层增加`overflow: hidden`可能暂时隐藏超出的部分，但也可能把正文或焦点轮廓裁掉。是否解决问题，要看内容与布局关系是否恢复，而不只是滚动条是否消失。

本章小结

正常文档流提供默认排列与随内容增长的基础。浮动让周围文字沿浮动盒子环绕，需要关注父容器高度与后续内容；`clear`与`flow-root`分别处理避让和容纳浮动等需求。

定位要同时判断参照与占位。相对定位保留原来的流中空间；绝对定位与固定定位不保留正常流占位；粘性定位随滚动受相关边界约束。覆盖问题还要观察层叠关系。

Flexbox在容器上组织一组项目。先找容器与直接子元素，再确定主轴和交叉轴，最后分析对齐、间距、基础尺寸、伸缩与换行。Grid与响应式布局将在CH05把这些认识扩展到整页的行列和宽窄屏适配。

复习与思考

1. 一个普通块级盒子的宽度变小，为什么后面的块级盒子通常不会自动排到右边？
2. 希望文字先沿图片旁边排列，再在图片下方恢复整行宽度，为什么适合使用浮动？这与始终保持左右两栏有什么不同？
3. 父容器只有浮动图片时，边框为什么可能没有包住图片？`flow-root`改变了什么？
4. `clear: both`会取消前面图片的浮动吗？它与`flow-root`的任务有什么不同？
5. 相对定位向下偏移20px后，后一个正常流段落是否一定随之下移20px？
6. HTML直接父元素为什么不一定是绝对定位元素的定位参照？
7. 给卡片加上`position: relative`，不设置偏移，是否仍然有用途？
8. 为什么卡片角标可能遮住标题？正文怎样给它留出空间？
9. 在本章简单页面中，固定定位和粘性定位分别观察哪个滚动参照？它们是否保留流中空间？
10. 为什么把`z-index`改成99999，也不保证元素出现在所有内容上方？
11. 给`nav`设置`display: flex`后，链接内部的`span`是不是也成为`nav`的弹性项目？
12. 从`row`切换为`column`，`justify-content`与`align-items`分别沿什么方向工作？
13. `gap`、`padding`和项目的`margin`分别表达哪一部分空间？
14. 为什么项目的基础尺寸不一定等于最终宽度？怎样通过页面或工具核对？
15. 文字换行与弹性项目换行有什么不同？Flexbox多行是否共享同一组列宽？
16. 为什么单独一行的信息项可能比上一行的信息项宽？

常见误解与代码审查

常见说法	需要修正的认识
盒子宽度小了，后面的块盒子就会补到旁边	尺寸与排列方式需要分别判断
浮动与绝对定位都是脱离正常流，因此效果相同	浮动会影响周围文字行的可用空间
父元素高度异常就设置一个固定高度	先检查内部内容如何参与高度计算
relative移动后，后面的内容会自动避让	原来的流中占位仍保留，可能产生重叠
absolute永远相对直接父元素	应查找实际建立包含块的对象
sticky就是另一种fixed	粘性定位保留流中空间，并受滚动区域和包含块边界约束
z-index越大就一定在全页最上面	数值比较受层叠上下文等条件约束
flex应当写在每个要横向排列的项目上	组织项目的规则通常写在共同容器上
justify-content就是水平对齐	它依据主轴方向工作
gap会给容器四周添加留白	外沿留白需由padding等规则表达
width或flex-basis就是项目最终宽度	还要考虑弹性分配和尺寸限制
wrap会让每一行的列自动严格对齐	各弹性行独立分配空间
页面不显示滚动条，就说明没有布局问题	内容可能被裁切、遮挡或挤到难以阅读

练习一 定位参照与占位

```
<section class="review-card">
  <span class="review-tag">新活动</span>
  <h2>校园读书交流会</h2>
</section>
```

```
.review-card { padding: 12px; border: 1px solid; }
.review-tag { position: absolute; top: 0; right: 0; }
```

标签没有待在卡片右上角。需要检查哪个参照？加一条定位声明后，标题是否一定不会被遮住？写出同时处理参照与空间的修改。

练习二 轴与对齐

一个高度明确为240px的容器使用`display: flex; flex-direction: column; justify-content: center; align-items: flex-start;`。项目在什么方向居中，在什么方向靠起点？如果把方向改成`row`，预测结果怎样变化？

练习三 宽度与换行

一组信息项在宽容器中位于同一行。缩窄容器后，项目挤到难以阅读。应该检查容器的哪条换行规则？增加一段介绍后，怎样区分文字换行与项目换行？请用页面或开发者工具验证，不要求手算最终宽度。

练习四 长内容与溢出

一个弹性项目内有很长的连续字符串。项目已经设置`flex: 1 1 0`，却仍把容器撑宽。应该检查哪些最小尺寸和内容换行规则？为什么只给外层加`overflow: hidden`不一定合适？

课后微任务 校园活动页面的布局改造

任务目标与起始状态

复制CH03页面或本章案例，用浮动、定位与Flexbox分别完成适合它们的局部任务。可以使用虚构资料，不填写真实联系方式。

必做要求

1. CSS保存在独立文件中，保留清楚的HTML内容顺序。
2. 在一处介绍中使用图片浮动和文字环绕，解释容器怎样包含浮动。
3. 在一张卡片中设置短角标，明确定位参照并避免遮挡标题。
4. 用Flexbox组织一组导航或操作链接，设置方向、对齐和`gap`。
5. 至少一组项目允许换行，解释项目基础尺寸与换行之间的关系。
6. 在布局实验页比较`relative`、`fixed`与`sticky`，记录一次滚动观察。
7. 用开发者工具完成一次尺寸或布局检查，制造并修复一处故障。
8. 用Tab键检查链接与表单控件，保留清楚的焦点提示。

本次任务不要求媒体查询或Grid；也不要求把固定、粘性定位全部放进活动首页。活动页中的静态表单继续保持提交禁用。

AI使用与审查

可以请AI解释一条规则、判断可能原因或建议局部修改。提问时给出相关HTML、CSS、预期与实际现象。接受修改前检查它改了哪一层容器、是否改变阅读顺序、是否以裁切内容掩盖问题。

提交时说明AI提供了什么帮助，以及自己通过哪一步观察确认结果。能够解释并局部修改规则，是采用代码之后需要承担的责任。

提交证据与验收

提交HTML、CSS、本地资源及简短记录：

```
我检查的容器与直接子元素：
浮动区域怎样包含图片：
角标的定位参照与预留空间：
Flexbox主轴方向及两种对齐的作用：
项目发生换行时的尺寸与间距依据：
我制造的故障、检查证据与修复：
键盘操作检查结果：
AI的帮助与我完成的验证：
```

验收时可随机改变一个布局方向、角标参照或项目基础尺寸。先预测，再修改，再通过页面和工具解释结果。

附录A 完整案例与运行

A.1 文件结构

```
src/ch04/
├── README.md
├── index.html          校园活动页面
├── layout-lab.html     浮动 定位与Flexbox独立实验
├── css/
│   ├── style.css       活动页样式
│   └── lab.css         实验页补充样式
└── images/
    └── activity.svg    本地教学插画
```

活动首页见index.html，完整CSS见style.css。独立实验见layout-lab.html及lab.css。实验页复用同目录的基础样式，不需要安装依赖。

A.2 打开与运行

可直接用浏览器打开两个HTML文件。也可以在`src/ch04`目录运行：

```
python3 -m http.server 8764
```

然后访问`http://localhost:8764/`与`http://localhost:8764/layout-lab.html`。所有图片来自本地，不依赖外部接口。

A.3 建议阅读顺序

活动首页将导航、角标、图文介绍、信息项和书目卡片组合在一起；阅读时可根据类名找到相应CSS。实验页把浮动容器、相对定位、角标参照、滚动标题和Flex尺寸分别隔离，便于判断各条规则的作用。

每次只改一个条件，记录实际盒子尺寸或位置。浏览器工具中的临时改动需要保存回文件，才会在刷新后继续生效。

附录B 属性速查与进一步阅读

B.1 浮动与定位速查

属性	常用值或用途
`float`	`none`、`left`、`right`
`clear`	`none`、`left`、`right`、`both`
`display: flow-root`	建立块格式化上下文，包含内部浮动等
`position`	`static`、`relative`、`absolute`、`fixed`、`sticky`
`top/right/bottom/left`	表达定位偏移或边界关系
`z-index`	在相应层叠规则下控制层级

B.2 Flexbox速查

设置对象	属性	本章常用值
容器	`display`	`flex`
容器	`flex-direction`	`row`、`column`
容器	`justify-content`	`flex-start`、`center`、`space-between`
容器	`align-items`	`stretch`、`flex-start`、`center`
容器	`flex-wrap`	`nowrap`、`wrap`
容器	`gap`	一个或两个间距值
项目	`flex`	增长 收缩 基础尺寸
项目	`align-self`	覆盖该项目的交叉轴对齐
项目	`min-width`	横向收缩场景中可检查是否需要`0`

B.3 识别旧代码中的clearfix

```
.clearfix::after {
  content: "";
  display: block;
  clear: both;
}
```

这个常见方案在容器末尾生成一个不含可见文字的伪元素，通过清除浮动，使其位置落到内部浮动下方，从而让容器正常流内容的高度延伸下去。了解原理即可，本章不用为了容纳浮动额外添加空HTML标签。

B.4 包含块与层叠上下文的其他情况

建立绝对定位包含块的因素不只有祖先的非`static`定位，`transform`等规则也可能参与。固定定位在特定祖先规则下，也可能不再依据视口。查错时要逐层检查实际样式，不能把本章简单例子的结论机械套用到所有页面。

定位元素配合非`auto`的`z-index`、`opacity`小于1、变换等，都可能建立层叠上下文；固定和粘性定位也有相应层叠行为。不同上下文作为整体参与外部比较。弹性项目设置非`auto`的`z-index`时也有自己的层叠规则，因此“所有元素都必须加position才能使用z-index”并不准确。

B.5 align-content与自动外边距

`align-content`用于控制多行弹性布局在交叉轴上如何分配行之间及两端的空间。要观察明显效果，需要允许多行布局，并在交叉轴上具有相应的可分配空间；它不能简单代替`align-items`。

在水平、从左到右的`row`布局中，给一个项目设置`margin-left: auto`，可以让自动外边距吸收正剩余空间，将该项目及后续内容推向右侧。自动外边距与`justify-content`有不同的作用阶段，先观察实际空间由谁吸收。

B.6 flex简写与收缩计算

`flex: none`相当于`0 0 auto`，`flex: auto`相当于`1 1 auto`。常见的`flex: 1`可以理解为从基础尺寸0参与增长并允许收缩的便捷写法；为了清楚展示三个组成部分，正文使用显式三值写法。在主轴尺寸不确定等场景中，百分比零基础与长度零基础可能存在差异，完整规则可查规范。

收缩分配不只看`flex-shrink`数值，还要结合项目的弹性基础尺寸。基础尺寸不同的两个项目，即使收缩因子相同，也不表示缩掉同样多的像素。实际算法还需处理最小、最大尺寸约束，本章不要求手工完成这些步骤。

附录C 自测参考与实践分析

C.1 核心问题参考

- 正常块布局不会因盒子变窄就自动将后续块盒子移到旁边；需要检查布局规则。
- 浮动让附近文字行避开图片及其外边距，超过这一区域后恢复较完整的行宽，适合图文环绕。固定的左右两栏是另一种排列关系。父容器需要包含浮动时可设置`flow-root`；某段后续内容需要从浮动下方开始时可设置`clear`。
- 相对定位保留原占位；绝对定位与固定定位不保留正常流占位。粘性定位保留流中的空间，并受相关边界约束。
- 绝对定位参照应查找实际包含块；卡片角标同时需要定位参照与不遮挡正文的空间安排。
- `justify-content`依据主轴，`align-items`依据交叉轴。改变方向后需要重新判断。
- Flexbox每条行分别分配空间；后一行项目可能增长到比前一行项目更宽。

C.2 代码审查参考

练习一可以先给`.review-card`设置`position: relative`，再增加足够的顶部内边距，例如`padding: 48px 12px 12px`，并给标签设置合理的内缩与尺寸。是否足够要结合字体、标签长度和放大后的结果验证；长标签宜考虑正常流。

练习二中，`column`的主轴竖直，项目组沿竖直方向居中，交叉轴向左侧起点对齐。改成`row`以后，主轴水平，项目组水平居中，交叉轴靠上。这里采用本章约定的书写方向。

练习三应先检查共同容器是否允许`flex-wrap: wrap`，再观察项目边框及Flex叠加显示。单个项目内的文字变成多行，不等于项目进入另一条弹性行；关闭换行后还可能出现收缩或溢出，应通过实际尺寸与内容验证。精确计算的例子见附录E.5。

练习四应检查弹性项目的自动最小宽度、内部图片或固定宽度子元素，以及长字符串的换行能力。视内容含义选择`min-width: 0`、图片尺寸限制或`overflow-wrap`等规则，而不是只裁掉可见溢出。

C.3 受控尺寸预测

假定容器内容宽度600px，三个项目没有边框、内外边距，均设置`min-width: 0; flex: 1 1 0`，项目间`gap: 12px`，且没有其他尺寸限制。三个项目共分配`600 - 24 = 576px`，每项192px。

把中间项目改成`flex: 2 1 0`，增长比例为1:2:1，结果为144px、288px、144px。这个例子成立依赖明确的基础尺寸0与相同的其他条件；不能据此推出任意两个项目只要增长因子为1和2，最终总宽度就一定为1:2。

C.4 排错顺序

先确认文件保存和CSS加载，再判断选择器是否匹配。随后核对容器与子元素关系、布局方式、定位参照或轴方向，最后检查尺寸限制、间距、换行、溢出和覆盖。

记录“原本预期、实际现象、检查证据、局部修改、再次验证”五项信息，能帮助区分已经解释清楚的问题与偶然消失的现象。

附录D 浮动的详细用法

正文第2部分介绍图文环绕的基本用法，并通过活动介绍案例组合使用`float`、`clear`与`flow-root`。本附录进一步解释文字行与段落盒子的区别、浮动对容器高度的影响，以及清除和包含浮动的细节，供核对代码和深入阅读时查阅。

D.1 float的取值与排列规则

`float`用于选择浮动方向，常用值如下：

值	含义
`none`	不浮动，采用该盒子原本的布局规则
`left`	向左侧浮动
`right`	向右侧浮动

浮动元素会尽量向指定一侧排列，并考虑容器边界及前面浮动的位置。这里的“侧”应结合所在布局区域理解，不是把元素移动到浏览器窗口边缘。

浮动盒子的尺寸、内外边距和边框都会影响周围可用空间。用于图文环绕的图片通常设置明确宽度，并在靠近文字的一侧留出外边距：

```
.reading-cover {
  float: left;
  width: 160px;
  height: auto;
  margin: 0 16px 8px 0;
}
```

这组规则使图片浮动到左侧，按原比例显示，并在右侧和下方分别留出16px、8px的距离。文字行在图片旁边使用剩余宽度；超过浮动外边距范围后，再恢复较完整的行宽。

为看清图片旁边还能排下多少文字，假定容器内容宽度为500px，图片宽160px、右外边距16px，图片没有额外边框或内边距，文字也没有其他水平缩进。图片旁边的可用行宽就是`500 - 160 - 16 = 324px`；超过图片及其下外边距范围后，文字又能使用500px的行宽。

图4-47 浮动的外边距也会占去文字旁边的空间

改为右浮动时，应同时检查留白方向；改为`none`时，图片恢复为非浮动内容。取消浮动与让后续段落避开浮动，是两个不同操作。

D.2 浮动对文字与容器高度的影响

浮动发生后，文字可以排到图片旁边，但这不表示整个段落盒子已经缩成右侧的一列。可以给段落增加背景，看清文字行与块盒子的区别：

```
.reading-intro p {
  margin: 0;
  border: 1px solid #205493;
  background-color: #e9f0f6;
}
```

在这组规则下，普通段落的块盒子可以延伸到浮动图片后方，而其中的文字行避开图片。背景、边框与文字的位置未必重合。浏览器处理的是“哪些文字行还剩多少宽度”，并没有把段落整体改造成固定宽度的右栏。

图4-48 文字绕开图片，段落盒子不一定缩成右栏

浮动也影响父容器高度的计算。将段落暂时移开，只在一个新容器中留下浮动图片：

```
<div class="float-box">
  <img class="float-image" src="images/activity.svg"
       alt="读书交流插画" width="800" height="300">
</div>
<p>这是容器后面的内容。</p>
```

```
.float-box {
  padding: 12px;
  border: 2px solid #205493;
}

.float-image {
  float: left;
  width: 180px;
  height: auto;
}
```

在这个例子中，父容器没有其他正常流内容，浮动图片也不按普通正常流子盒子的方式撑开父容器。父容器的自动内容高度为零，只有上下内边距和边框仍然形成可见高度，图片会伸到边框外。

图4-49 只有浮动子元素时，父容器可能包不住图片

如果容器里本来有很多正常流文字，文字形成的高度可能已经足够大，图片伸出的现象就不明显。只保留浮动图片，可以把“文字撑开的高度”与“图片是否计入高度”区分开。附录D.4将让父容器的高度包含内部浮动。

D.3 使用clear清除后续内容的浮动影响

图文介绍后面还可能有一句总结，例如“欢迎带着问题参加”。这句话希望从图片下方另起，不继续围绕图片排列。

```
<div class="reading-intro">
  <img class="reading-cover" src="images/activity.svg"
       alt="读书交流插画" width="800" height="300">
  <p>选择一本喜欢的书。</p>
  <p class="clear-summary">欢迎带着问题参加。</p>
</div>
```

```
.clear-summary {
  clear: both;
}
```

`clear`用于控制盒子与前面浮动的避让关系。`left`要求避开相关左浮动，`right`要求避开相关右浮动，`both`则同时考虑两侧。它会在需要时增加向下的避让，使这段总结从浮动盒子下方开始。

这条声明写在总结段落上，处理的是总结应从哪里开始。图片仍然采用`float: left`，前面的介绍仍然可以环绕图片。只有把图片自己的`float`改为`none`，才是取消它的浮动。

图4-50 清除浮动影响与取消图片浮动是两件事

如果前面的文字本来已经超过图片高度，总结可能已经在图片下方，增加`clear`后看不出明显变化。判断属性是否生效时，要同时看浮动范围与当前内容位置，不能只凭画面有没有移动作结论。

D.4 使用flow-root让容器包含浮动

附录D.2的`.float-box`只包含一张浮动图片，边框没有包住图片。若希望这个区域成为一个完整单元，让后面的内容从它下方继续排列，可以让父容器建立独立的块格式化上下文：

```
.float-box {
  display: flow-root;
  padding: 12px;
  border: 2px solid #205493;
}
```

这里保留原有边框和内边距，只增加`display: flow-root`。浏览器会把内部浮动纳入这个容器自动高度的计算范围，因此边框能够包住图片，容器后面的正常流内容也会排到整个区域之后。

图片原始尺寸为800×300，按180px宽等比例显示时，高度为`180 × 300 ÷ 800 = 67.5px`。父容器原来的内容高度为零，边框外沿高度为`0 + 12 × 2 + 2 × 2 = 28px`；包含浮动以后，边框外沿高度变为`67.5 + 24 + 4 = 95.5px`。这里图片没有额外上下外边距，父容器也没有其他影响高度的内容。

图4-51 flow-root让自动高度包含内部浮动

块格式化上下文通常简称BFC，可以先理解为具有自身块布局规则的一块区域。`flow-root`明确建立这样的区域。它没有取消图片的浮动；如果容器内部还有段落，文字依然能够围绕图片。

在完整介绍区域中，也可以把同样的规则用于图片和段落的共同父元素：

```
.reading-intro {
  display: flow-root;
}
```

现在，图文介绍区域既包含图片，也包含文字；后面的时间地点信息不会继续受到这里浮动图片的环绕影响。

需要区分两个问题：`clear`让指定的后续盒子避开前面的浮动，`flow-root`让一个容器建立独立的块布局区域并包含内部浮动。它们的设置对象和作用范围不同。旧代码中常见的clearfix写法见附录B。

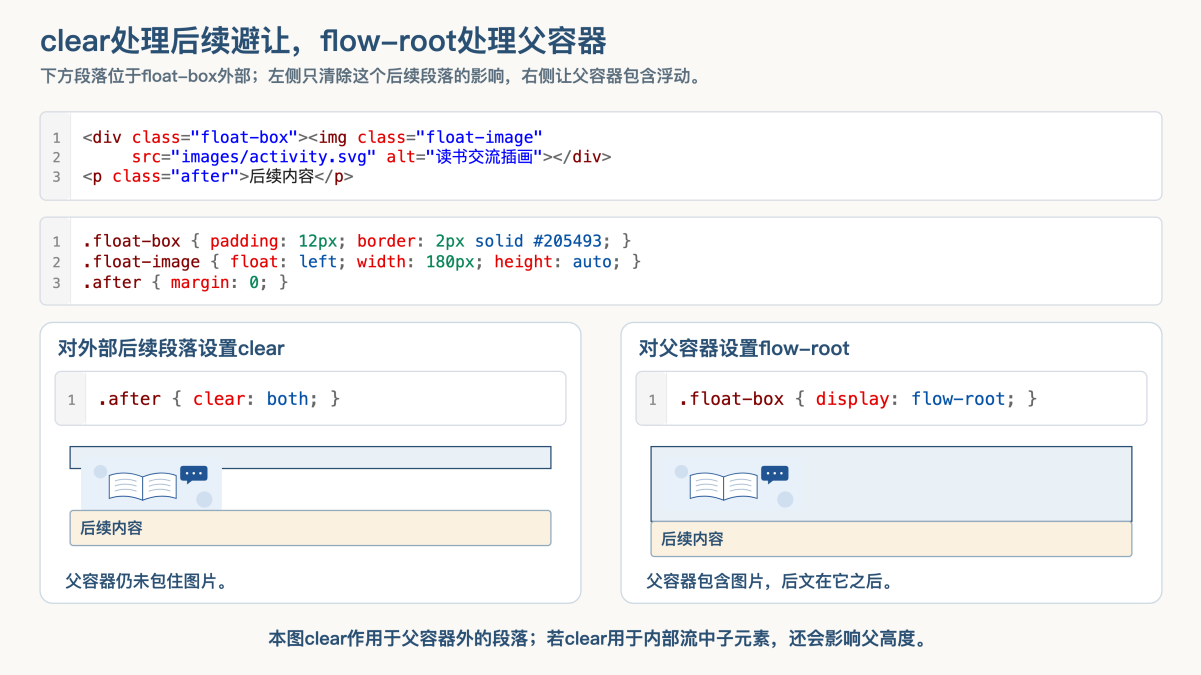

图4-52 clear处理后续避让，flow-root处理父容器

附录E Flexbox的详细用法

本附录按对齐、间距、弹性尺寸和换行逐项展开，供需要深入理解或核对代码时查阅。正文第4.3节介绍常见布局的基本用法；这里保留更多取值、受控计算和对照图。

E.1 justify-content与主轴对齐

主轴对齐处理项目组在主轴上的位置，以及剩余空间在项目之间和两端的分配。容器属性`justify-content`表达这组规则，它通常不直接改变项目自身的宽度。

三个导航链接排成一行以后，容器通常还剩一些空间。希望它们整体居中，可以让浏览器把这些剩余空间分到项目组两侧：

```
.site-nav {
  display: flex;
  justify-content: center;
}
```

`justify-content`控制主轴上的对齐与空间分配。这里没有让各链接主动增长，因此可以把三个链接看成一组：`center`将项目组放在主轴中间，两端各保留一部分空白。

假设容器内容宽600px，三个项目边框盒宽度均为100px，且没有外边距和`gap`，项目总宽为300px，剩余300px。采用`center`时，两端各留150px；采用`flex-start`时，项目靠主轴起点排列，剩余空间集中在末端。

图4-53 主轴对齐：项目宽度相同，剩余空间放在哪里

如果改为`space-between`，两端项目靠近容器的起点和终点，300px剩余空间分到两个项目间隔中，每个间隔150px。项目本身仍然是100px，并没有被拉宽。

值	主轴上的基本结果
`flex-start`	项目组靠起点
`flex-end`	项目组靠终点
`center`	项目组居中
`space-between`	剩余空间分到项目之间
`space-around`	每个项目两侧分配空间，外沿间距为内部间距的一半
`space-evenly`	项目之间与两端分配相等的空间

图4-54 分散对齐：比较项目之间与两端的空间

这些结果需要结合可分配的空间理解。如果项目已经通过增长用完剩余宽度，改变`justify-content`就未必有明显效果。若采用`column`，主轴变成竖直方向；容器高度只是随内容增长时，也可能没有多余高度让项目组显出居中效果。

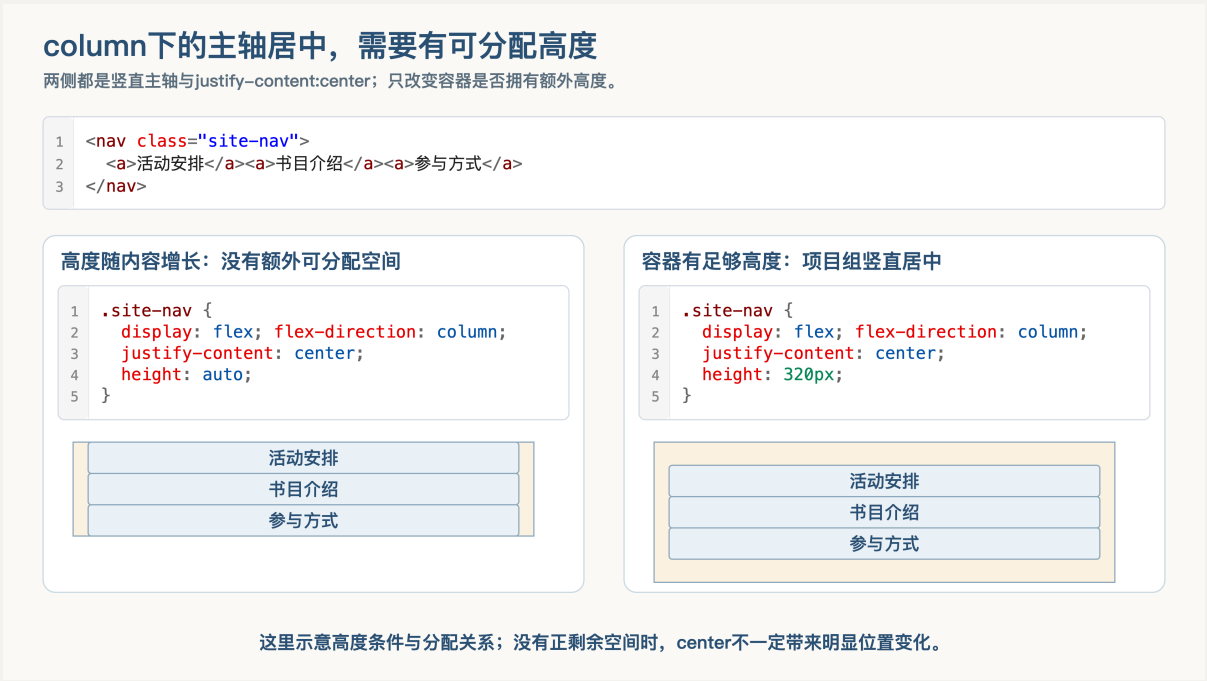

图4-55 column下的主轴居中，需要有可分配高度

E.2 align-items与交叉轴对齐

交叉轴对齐处理同一弹性行内各项目在交叉方向上的位置。容器属性`align-items`给出共同规则；单个项目需要采用其他规则时，可以通过`align-self`覆盖。

同一行项目的高度不一定相同。活动信息中，一个项目可能只有“周五下午”，另一个项目可能包含两行地点说明。希望这些项目沿另一个方向居中，可以设置`align-items`：

```
<div class="info-row">
  <div>周五下午</div>
  <div>图书馆<br>交流空间</div>
  <div>欢迎参加</div>
</div>
```

```
.info-row {
  display: flex;
  align-items: center;
}

.info-row > div {
  padding: 8px 16px;
}
```

在默认`row`方向下，交叉轴是竖直方向。较高项目撑起所在弹性行的高度，`align-items: center`使较矮项目在这一行的交叉轴空间中居中，所以它们的上边沿不一定相同，但中线对齐。

改为`flex-start`，项目靠交叉轴起点，表现为上边沿对齐；改为`flex-end`，则靠交叉轴终点。`baseline`依据项目的基线进行对齐，常用于比较不同字号的文字。

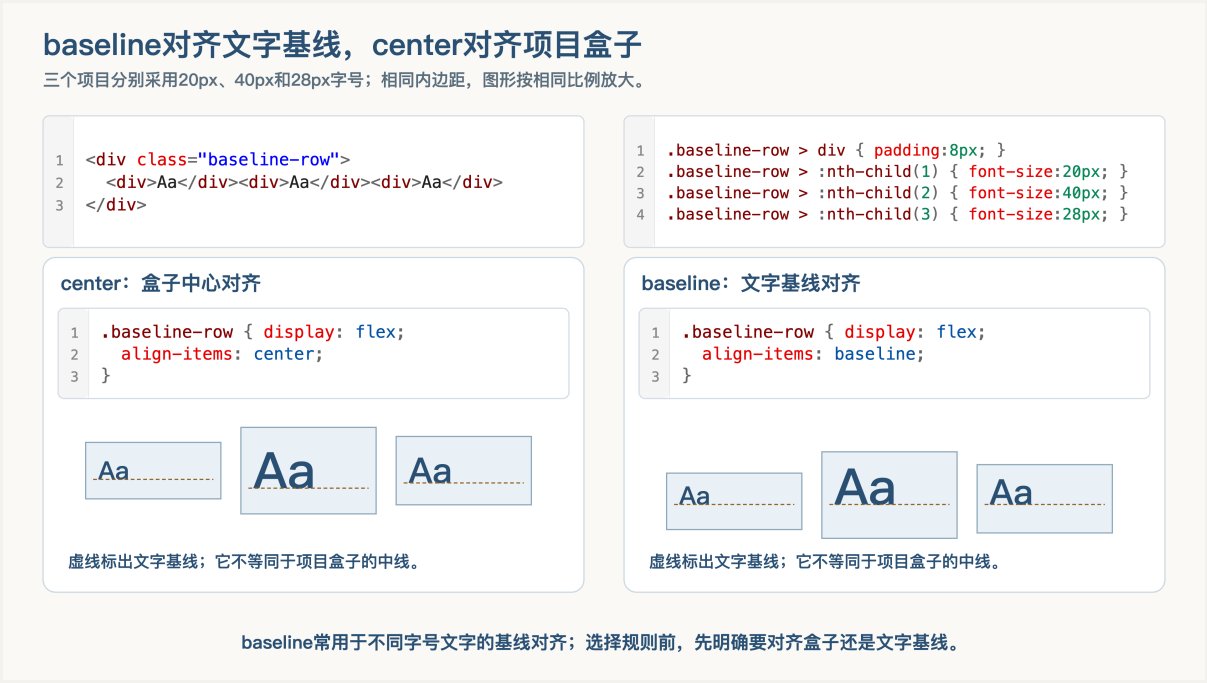

图4-56 baseline对齐文字基线，center对齐项目盒子

`stretch`在交叉轴尺寸为自动等条件满足时，会拉伸项目来占据所在行的交叉轴空间。在本例中，较矮项目可以被拉到同一行的高度。若已经给项目明确设置了高度，就不能再无条件期待它被拉成一样高。

图4-57 stretch有尺寸条件，align-self只调整一个项目

这里对齐的是项目盒子。项目内部的文字怎样排列，还要根据行高、`text-align`或内部布局判断。将`align-items`改为`center`，不会自动把每个盒子里的文字也变成水平居中。

E.3 gap与项目间距

对齐决定项目怎样使用剩余空间，间距则用来表达项目之间应留出的距离。先用一个简单容器区分内部边缘留白与项目间距：

```
.list {
  display: flex;
  padding: 32px;
  gap: 24px;
}

.item {
  margin: 0;
}
```

`padding: 32px`在容器内容与边框之间留出空间；`gap: 24px`在相邻项目之间形成间隔。第一项与容器边缘之间的留白来自`padding`，不会因为设置了`gap`就再自动增加一个同样的边缘间隔。

图4-58 padding在容器边缘，gap在相邻项目之间

当容器允许项目分成多行时，还可以分别设置两个方向的间距：

```css
.site-nav {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 8px 16px;
}
```

在这里的`row`布局中，前一个值8px表示弹性行之间的间距，后一个值16px表示同一行项目之间的间距。只写一个值时，两者采用相同数值。`flex-wrap: wrap`允许建立多条行，附录E.5会专门说明。

项目自身的`margin`仍然会参与布局。若从CH03迁移导航时保留链接原来的`margin: 4px`，再增加`gap`，可见距离就会包含两种来源。通常应先把链接外边距重设为零，再用容器统一表达项目间距。

`gap`也不总等于最终看见的全部距离。例如同时使用`space-between`时，项目间还会分得额外剩余空间。分析距离时，要区分容器留白、项目外边距、`gap`与对齐分配。

图4-59 双值gap区分行与项目，margin仍会额外参与

E.4 flex-basis flex-grow与flex-shrink

导航通常按内容决定宽度，时间地点信息则可能希望均匀分配一行。Flexbox不仅决定排列方向，还能在项目之间分配可用空间。

先建立一个便于计算的例子：容器内容宽600px，内部有A、B、C三个项目，项目之间的间距为12px。项目没有边框、内边距和外边距，也没有其他最小、最大尺寸限制。

```
<div class="size-row">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>
```

```
.size-row {
  display: flex;
  width: 600px;
  gap: 12px;
}

.size-row > div {
  flex: 1 1 0;
  min-width: 0;
}
```

`flex`的三值简写依次表示增长因子、收缩因子、基础尺寸。这里`1 1 0`表示从基础尺寸0开始参与分配，允许按相同增长因子取得剩余空间，也允许在需要时收缩。

图4-60 flex三值：增长因子、收缩因子、基础尺寸

三个项目之间有两个间隔，先占去`12 × 2 = 24px`。可分给项目的空间为`600 − 24 = 576px`。三个项目基础尺寸均为零、增长因子均为1，所以每项得到`576 ÷ 3 = 192px`。

只把中间项目的增长因子改为2：

```
.size-row > div:nth-child(2) {
  flex-grow: 2;
}
```

`:nth-child(2)`在这里选中第二个子元素。576px剩余空间改为按1:2:1分成四份，每份144px，三个项目分别得到144px、288px和144px。中间项目在这个例子中恰好是两侧项目的两倍，原因是它们采用了共同的基础尺寸0和明确的其他条件。

图4-61 基础尺寸为0：按增长因子分配576px

属性	在空间分配中的作用
`flex-basis`	提供主轴上的基础尺寸
`flex-grow`	按增长因子分配正剩余空间
`flex-shrink`	空间不足时，与基础尺寸等一起决定收缩分配

项目也可以从非零尺寸开始。例如，`flex: 1 1 180px`从180px基础参与分配；`flex: 0 0 220px`不增长、不收缩，以220px作为基础。增长因子描述的是增长空间的分配比例，不能普遍当作最终总宽度的比例。

未显式设置`flex`时，初始组合是`0 1 auto`：不主动增长，允许收缩，基础尺寸由主轴尺寸属性或内容等决定。因此，弹性项目上写出的`width`不一定等于最终宽度。

图4-62 从180px开始增长：1:2:1不等于最终宽度比

本例的`min-width: 0`允许项目在需要时缩小到默认内容最小宽度以下。它不负责让长字符串自动换行；实际页面还要检查文字的换行规则与图片尺寸。收缩分配的补充说明及常用简写见附录B.6；完整算法可查阅规范。

E.5 flex-wrap与项目换行

另建一组A、B、C项目，从180px基础尺寸开始，容器仍然有12px间距。这里使用独立的`wrap-row`类，三个项目都采用相同的增长与收缩因子：

```
<div class="wrap-row">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>
```

```
.wrap-row {
  display: flex;
  width: 600px;
  gap: 12px;
  flex-wrap: wrap;
}

.wrap-row > div {
  flex: 1 1 180px;
  min-width: 0;
}
```

`flex-wrap`决定容器是否允许建立多条弹性行。初始值`nowrap`要求项目留在单行；`wrap`允许放不下的项目进入下一行。这里继续使用没有项目边框、内外边距和其他尺寸限制的受控例子。

在600px容器中，三个180px项目与两个间隔共需`180 × 3 + 12 × 2 = 564px`，可以放进一行。还剩36px，三个项目的增长因子相同，各增加12px，所以最终仍是每项192px。

将容器内容宽度减为550px时，564px的排列需求已经放不下。启用`wrap`后，第三个项目进入下一行。第一行两个项目再分配这一行的剩余空间，第二行的项目也只依据自己所在行分配空间。

如果只把`wrap`改为`nowrap`，浏览器会尝试让三个项目留在一行，并按照收缩规则调整它们。在本例内容允许缩小、三个项目基础尺寸和收缩因子相同的条件下，项目可用空间为`550 − 24 = 526px`，每项最终约为`526 ÷ 3 = 175.33px`。

这两种结果说明：允许换行后，浏览器会先按相关基础尺寸和限制决定哪些项目组成一行，再在每行内部进行弹性分配；不能先假设项目可以任意缩小，就认为永远不必换行。实际页面若存在长字符串、图片或最小宽度限制，不允许换行时也仍可能溢出。

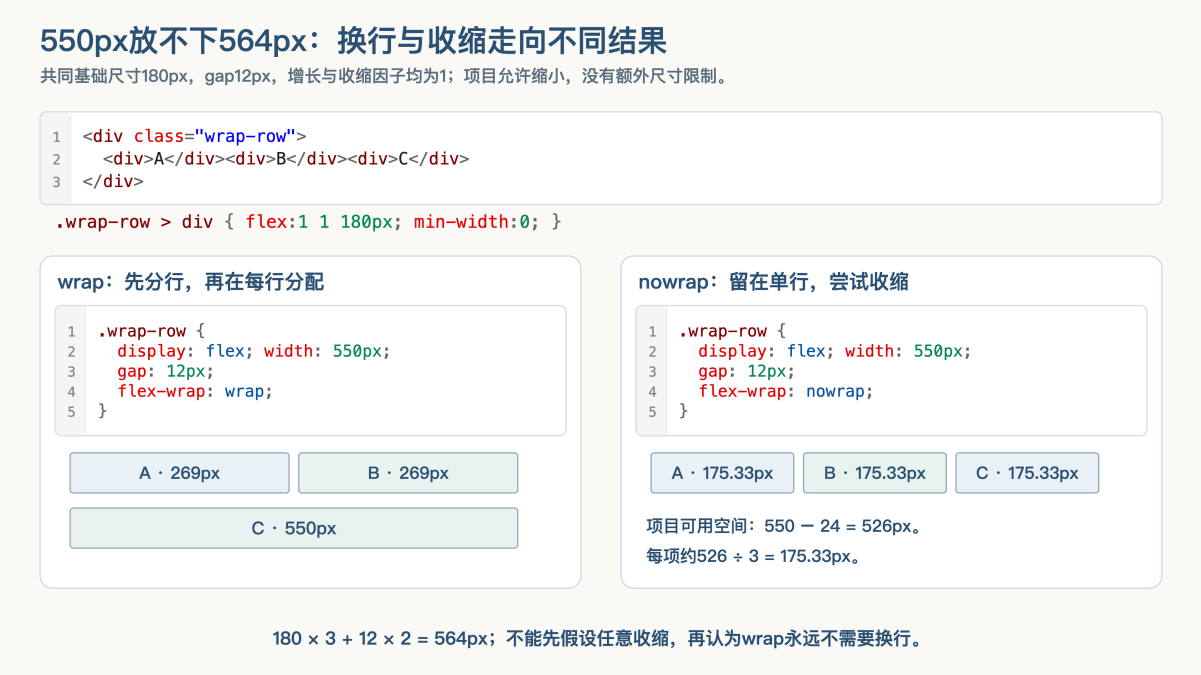

图4-63 550px放不下564px：换行与收缩走向不同结果

此外，项目换行与文字换行是两回事。一个项目里的句子变成三行，仍然可能只是一个弹性项目；第三张卡片整体移到下一条弹性行，才是这里讨论的项目换行。

E.6 多行布局与逐行空间分配

上一节中，550px容器将三个项目分成两行。第一行两个项目，第二行只有一个项目，最终会得到怎样的宽度？

第一行只有一个12px间隔，两个项目共可使用`550 - 12 = 538px`。每个项目的基础尺寸是180px，增长因子相同，因此最终各占269px。

第二行只有一个项目，没有项目间隔。它的基础尺寸仍为180px，也允许增长，所以会取得这一行的剩余空间，最终占550px。

```
第一行：269px + 12px + 269px = 550px
第二行：550px
```

三个项目采用同一条`flex`规则，却可能得到不同宽度，因为它们所在的弹性行不同。**每条弹性行独立分配空间**，不会为了与前一行某个项目对齐，就自动保持同一组列宽。

图4-64 相同flex规则，不同行也会得到不同宽度

这也是为什么一组卡片从三列变成两列后，最后一张可能铺满整行。若设计要求最后一张仍沿用前面各列的宽度，或要求多行之间共享明确的列关系，就需要继续考虑Grid等布局方式，CH05会用同一组书目比较。

多行布局还有另一层对齐：`align-items`控制每条行内部项目的交叉轴对齐，`align-content`控制这些行在交叉轴剩余空间中怎样分布，后者见附录B。

E.7 使用开发者工具核对详细尺寸

在页面中右键检查导航，可以从DOM结构和实际计算值验证前面的解释。选中`nav`后，先确认Styles中确实有生效的`display: flex`，再查看浏览器提供的Flex布局标记或叠加显示。

检查位置	观察对象	可以解释的现象
Elements / 元素	容器的直接子元素	为什么这一层盒子成为项目
Styles / 样式	方向、对齐、gap与换行声明	当前采用哪些布局条件
Computed / 计算样式与盒模型	项目实际尺寸和边距	为什么最终宽度不同于基础尺寸
Flex布局叠加显示	轴、弹性行与间隔	是项目换行还是内部文字换行

例如，在布局实验页选中`.wrap-row`，把内容宽度设为550px。保持`wrap`时，检查第一行两项和第二行一项；改为`nowrap`后，再检查每项是否收缩。这样就把附录E.5的两种解释对应到了实际盒子。

图4-65 用开发者工具核对：结构、声明与实际尺寸

活动页中的卡片还包含内边距与边框，测量时不能直接套用无边框计算图的数字。应先记录容器内容宽度，再核对项目采用的`box-sizing`、基础尺寸和额外空间。

开发者工具允许临时开关或修改声明，适合验证原因。确认修改有效以后，要保存回CSS源文件；否则刷新页面，浏览器仍会重新读取原来的规则。

参考资料

- MDN CSS布局入门：进一步了解常见布局方式。
- MDN 浮动：查阅浮动及图文环绕。
- MDN position：查阅各类定位与示例。
- MDN Flexbox基本概念：复习容器、项目与轴。
- W3C CSS视觉格式化模型：正常流、浮动与早期定位规则。
- W3C CSS定位布局：定位参照与定位行为。
- W3C CSS弹性盒布局：弹性尺寸、排列与空间分配的完整规则。